

第 12 周：MDP 与 Tabular RL 理论

从 Bandit 到序贯决策，收敛性理论的完整图景

优化算法课程讲义

摘要

上周我们学习了 Bandit 问题：每轮选一个 arm，拿到奖励，选择之间相互独立。本周我们加入状态：你的选择不仅得到奖励，还会改变世界状态，影响未来的选择。这就是马尔可夫决策过程 (MDP)。我们将证明 Bellman 方程为何成立、值迭代为何收敛，以及各种 RL 算法的收敛性理论。

目录

1 从 Bandit 到 MDP：一步之遥	4
2 MDP 形式化定义	5
3 Bellman 方程：递归的威力	7
3.1 Bellman 期望方程	7
3.2 Bellman 最优方程	8
4 值迭代：收缩映射的魔力	9
4.1 收缩映射与 Banach 不动点定理	9
5 策略迭代：更快的收敛	11
6 Monte Carlo 方法：最朴素的 model-free 方法	12
6.1 Bandit 的直接推广	13
6.2 MC 评估算法 (prediction)	13
6.3 MC 的统计性质	14
6.4 MC control: 评估 + 改进的循环	14
6.5 MC 的两个软肋	15
7 从 Bandit Regret 到 RL Regret	16
8 UCB-VI: 把 UCB 推广到 MDP	17
8.1 UCB-VI 的 Regret 分析	18
8.2 Regret 下界	23

9	时序差分学习：从 prediction 到 control	24
9.1	Prediction 问题：给定策略，估计价值	24
9.2	TD(0)：用 bootstrap 替换 G_t	25
9.3	从 prediction 到 control：为什么必须学 Q 不学 V	27
9.4	SARSA：on-policy TD control	28
9.5	Q-learning：off-policy TD control	29
10	核心概念与工程技巧：on/off-policy、经验回放、目标网络	32
10.1	On-policy vs off-policy：精确定义	32
10.2	重要性采样：off-policy 的统计桥梁	33
10.3	Online vs offline：另一个独立维度	34
10.4	经验回放：让 online 算法重复消化样本	35
10.5	目标网络：bootstrap 的稳定化器	36
10.6	Q-learning 的过估计偏差与 Double Q-learning	37
10.7	探索策略小结	38
10.8	Rollout：把”模拟一段轨迹”独立成一个动词	39
10.9	其他小技巧速览	41
10.10	算法地图：把这一切收进一张表	45
11	Q-learning 收敛性	45
11.1	有模型 vs 无模型：核心区别	46
11.2	Q-learning 算法	49
12	线性函数逼近的收敛性	52
12.1	Linear TD(0)	52
12.2	Deadly Triad：发散的危险	53
13	两个关键引理	54
13.1	Simulation Lemma：模型误差如何传播	54
13.2	为什么 Model-Based RL 在实践中常常不如 Model-Free?	55
13.3	Performance Difference Lemma	57
14	Multi-Agent RL 的收敛性	58
14.1	零和博弈：Minimax 收敛	59
14.2	一般和博弈：收敛困难	61
15	TD(λ) 与资格迹	61
16	LSTD/LSPI：批量方法	63
17	平均奖励 MDP	64
18	GPU 上的动态规划	65

1 从 Bandit 到 MDP：一步之遥

回顾上周

上周的 Bandit 问题：

- K 台老虎机，每台有未知的期望奖励 μ_a
- 每轮选一台拉，观察奖励
- 选择之间**完全独立**——你选什么不影响下一轮的世界

我们学了 UCB（乐观原则）和 Thompson Sampling（概率匹配），Regret 都是 $O(\sqrt{T})$ 。

本周的扩展

现实中，决策往往**改变世界状态**：

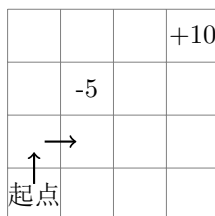
- 下棋：每步棋改变棋盘局面
- 开车：每个动作改变位置和速度
- 治疗：每次用药改变病人状态

MDP 就是在 Bandit 基础上加一个东西：**状态转移**。

	Bandit	MDP
状态	只有 1 个（或者说没有）	多个状态 $s \in S$
动作效果	只影响当前奖励	影响奖励 + 转移到新状态
决策	独立的	序贯的，相互影响

例 1.1 (从老虎机到 GridWorld). **Bandit**: 3 台老虎机，选一台拉，得奖励，结束。下一轮还是面对同样的 3 台机器。

MDP (GridWorld): 你在一个 4×4 网格里，每个格子是一个状态。



- 你在某个格子（状态 s ）
- 选择一个方向走（动作 a ）
- 可能滑到旁边（随机转移 $P(s'|s, a)$ ）
- 到达新格子，得到那个格子的奖励（ $R(s, a)$ ）

- 现在你在**新状态** s' ，面对的选择变了！

这就是 Bandit 和 MDP 的核心区别：**你的选择改变了你面对的世界。**

2 MDP 形式化定义

定义 2.1 (马尔可夫决策过程 (MDP)). MDP 是一个五元组 (S, A, P, R, γ) :

- S : 状态空间 ($|S|$ 个状态)
- A : 动作空间 ($|A|$ 个动作)
- $P: S \times A \times S \rightarrow [0, 1]$: 转移概率, $P(s'|s, a)$
- $R: S \times A \rightarrow \mathbb{R}$: 奖励函数
- $\gamma \in [0, 1)$: 折扣因子

例 2.1 (咖啡店选址——最简单的 MDP). 你要决定每天去哪家咖啡店。

状态 S : 你现在的心情 { 开心, 一般, 烦躁 }

动作 A : 去哪家店 { 星巴克, 瑞幸 }

转移 P :

- 心情一般 + 去星巴克 (贵但好喝) $\rightarrow$ 70% 变开心, 30% 还是一般
- 心情一般 + 去瑞幸 (便宜但一般) $\rightarrow$ 40% 变开心, 60% 还是一般
- 心情烦躁 + 去星巴克 $\rightarrow$ 50% 变一般, 50% 还是烦躁

奖励 R : 开心时 +2, 一般时 +1, 烦躁时-1

问题: 长期来看, 应该怎么选咖啡店?

马尔可夫性质

”马尔可夫”的意思是：**未来只依赖于现在，不依赖于过去。**

生活例子: 天气预报

”明天是否下雨”只取决于”今天的天气状况”，不取决于”上周下了几天雨”。

如果今天是晴天，明天 30% 下雨。不管之前连续晴了 3 天还是 30 天，这个概率都是 30%。这大大简化了问题：我们只需要记住”现在在哪”，不需要记住”怎么来的”。

例 2.2 (为什么需要折扣因子 γ). 假设你可以选择:

- 选项 A: 今天拿 100 块
- 选项 B: 一年后拿 100 块

大多数人选 A! 为什么?

1. **不确定性:** 一年后的事谁知道呢

2. 机会成本：今天的 100 块可以投资

3. 即时满足：人类天生更看重眼前

γ 就是“未来的钱打几折”。 $\gamma = 0.9$ 意味着：

- 明天的 1 块 = 今天的 0.9 块
- 后天的 1 块 = 今天的 $0.9^2 = 0.81$ 块
- 一年后的 1 块 = 今天的 $0.9^{365} \approx 0$ 块

定义 2.2 (策略与值函数). **策略** $\pi : S \rightarrow A$ (或 $\pi : S \rightarrow \Delta(A)$) 告诉我们在每个状态选什么动作。

值函数 $V^\pi(s)$ ：从状态 s 出发，按策略 π 行动，获得的期望累积折扣奖励：

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right]$$

动作-值函数 $Q^\pi(s, a)$ ：从状态 s 采取动作 a ，然后按 π 行动：

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s')$$

例 2.3 (值函数的直觉——房产估值). 把 $V(s)$ 想象成房子的价值：

- 状态 s = 房子的位置
- 动作 a = 装修方案
- 奖励 R = 每月租金收入
- 转移 P = 装修后房子状态变化（可能升值也可能出问题）

$V(s)$ 就是“这个位置的房子，长期能给我带来多少租金收入（折现到今天）”。
好位置（市中心） V 高，差位置（偏远郊区） V 低。

V 和 Q 的关系

$V^\pi(s) = Q^\pi(s, \pi(s))$ （确定性策略）

生活比喻：

- $V(s)$ = 在状态 s ，按我的习惯做事，能得多少分
- $Q(s, a)$ = 在状态 s ，如果这次选动作 a ，能得多少分
- $V(s) = Q(s, \text{我的习惯选择})$

数值例子：状态 s ，动作 a ，奖励 $R(s, a) = 1$ ， $\gamma = 0.9$ 。

转移：50% 到 s_1 ($V^\pi(s_1) = 5$)，50% 到 s_2 ($V^\pi(s_2) = 3$)。

$$Q^\pi(s, a) = \underbrace{1}_{\text{立即奖励}} + \underbrace{0.9}_{\text{折扣}} \times \underbrace{(0.5 \times 5 + 0.5 \times 3)}_{\text{未来期望价值}} = 1 + 3.6 = 4.6$$

3 Bellman 方程：递归的威力

大白话

Bellman 方程是 RL 的牛顿定律，一切都从它推导出来。

核心思想极其简单：今天的价值 = 今天的奖励 + 明天的价值（打折）

这个递归关系把无限期问题变成了一步一步的计算。

例 3.1 (Bellman 方程的生活版——职业规划). 你在考虑职业发展，当前状态是”初级工程师”。

问：这个状态的”价值”是多少？（未来能赚多少钱）

Bellman 式思考：

$$V(\text{初级}) = \underbrace{15\text{万}}_{\text{今年工资}} + 0.9 \times \underbrace{[\text{明年状态的价值}]}_{\text{可能升职/跳槽/原地}}$$

如果 30% 概率升职（高级值 50 万），70% 概率还是初级：

$$V(\text{初级}) = 15 + 0.9 \times (0.3 \times 50 + 0.7 \times V(\text{初级}))$$

解方程： $V = 15 + 13.5 + 0.63V$ ，得 $V(\text{初级}) = 77$ 万。

这就是 Bellman 方程：当前价值 = 即时回报 + 折扣后的未来期望！

3.1 Bellman 期望方程

定理 3.1 (Bellman 期望方程). 对任意策略 π ：

$$V^\pi(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right]$$

用算子形式： $V^\pi = T^\pi V^\pi$ ，即 V^π 是 Bellman 算子 T^π 的不动点。

例 3.2 (2 状态 MDP——学生的一天). 状态：{学习中, 摸鱼中}

转移（无动作，自动转移）：

- 学习中 $\rightarrow$ 60% 继续学习，40% 开始摸鱼
- 摸鱼中 $\rightarrow$ 30% 开始学习，70% 继续摸鱼

奖励：学习 +2 分，摸鱼 +1 分。 $\gamma = 0.9$

Bellman 方程：

$$V(\text{学习}) = 2 + 0.9 \times (0.6 \times V(\text{学习}) + 0.4 \times V(\text{摸鱼}))$$

$$V(\text{摸鱼}) = 1 + 0.9 \times (0.3 \times V(\text{学习}) + 0.7 \times V(\text{摸鱼}))$$

求解：从第二个方程整理： $V(\text{摸鱼}) = 1 + 0.27V(\text{学习}) + 0.63V(\text{摸鱼})$

$$0.37V(\text{摸鱼}) = 1 + 0.27V(\text{学习}) \rightarrow V(\text{摸鱼}) = 2.7 + 0.73V(\text{学习})$$

$$\text{代入第一个: } V(\text{学习}) = 2 + 0.54V(\text{学习}) + 0.36(2.7 + 0.73V(\text{学习}))$$

$$V(\text{学习}) = 2.97 + 0.8V(\text{学习}) \rightarrow V(\text{学习}) = 14.85$$

$$V(\text{摸鱼}) = 2.7 + 0.73 \times 14.85 = 13.54$$

结论：学习状态更”值钱”！（14.85 vs 13.54）

Bellman 期望方程的证明. 从定义展开:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right] \\ &= \mathbb{E}_\pi [R(s_0, a_0) \mid s_0 = s] + \gamma \mathbb{E}_\pi \left[\sum_{t=1}^{\infty} \gamma^{t-1} R(s_t, a_t) \mid s_0 = s \right] \end{aligned}$$

第一项: $\mathbb{E}_\pi [R(s_0, a_0) \mid s_0 = s] = \sum_a \pi(a|s) R(s, a)$

第二项: 利用马尔可夫性, 给定 s_1 , 未来与 s_0 独立:

$$\mathbb{E}_\pi \left[\sum_{t=1}^{\infty} \gamma^{t-1} R(s_t, a_t) \mid s_0 = s \right] = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) V^\pi(s')$$

合并得证。 □

3.2 Bellman 最优方程

定义 3.1 (最优值函数). $V^*(s) = \max_\pi V^\pi(s)$, $Q^*(s, a) = \max_\pi Q^\pi(s, a)$

定理 3.2 (Bellman 最优方程).

$$\begin{aligned} V^*(s) &= \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right] \\ Q^*(s, a) &= R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a') \end{aligned}$$

例 3.3 (期望 vs 最优——午餐选择). 你饿了, 两个选择:

- 动作 A: 食堂 (确定 7 分满意度)
- 动作 B: 新餐厅 (50% 概率 9 分超好吃, 50% 概率 5 分踩雷)

如果你的策略是”随机选”(各 50%):

$$V^\pi = 0.5 \times 7 + 0.5 \times (0.5 \times 9 + 0.5 \times 5) = 0.5 \times 7 + 0.5 \times 7 = 7$$

最优策略: 比较两个动作的期望

- $Q^*(s, \text{食堂}) = 7$
- $Q^*(s, \text{新餐厅}) = 0.5 \times 9 + 0.5 \times 5 = 7$

两个一样好! $V^* = 7$ 。

但如果新餐厅是 60% 概率 9 分: $Q^*(s, \text{新餐厅}) = 7.4 > 7$

那最优策略就是总是选新餐厅, $V^* = 7.4$ 。

期望 vs 最优

Bellman 期望方程: 按固定策略 π 行动的价值, 用 $\sum_a \pi(a|s)$ 加权

Bellman 最优方程: 每一步都选最好的, 用 $\max_a$

最优策略就是贪婪策略: $\pi^*(s) = \arg \max_a Q^*(s, a)$

4 值迭代：收缩映射的魔力

大白话

知道 Bellman 方程是不动点后，怎么找到它？

最简单的方法：**迭代**！从任意初始值开始，反复应用 Bellman 更新。

为什么能行？因为 Bellman 算子是**收缩映射**——每次迭代都让你更接近真解！

例 4.1 (收缩映射的生活比喻——调节水温). 你想把浴缸水温调到 40°C ，但你不知道现在多少度。

策略：每次加一点热水或冷水，让温度往 40°C 靠近。

如果你的调节方式是”每次让温度往 40°C 移动一半”：

- 现在 60°C $\rightarrow$ 加冷水 $\rightarrow$ 变成 50°C (离目标近了一半)
- 现在 50°C $\rightarrow$ 继续调 $\rightarrow$ 变成 45°C
- 现在 45°C $\rightarrow$ 继续调 $\rightarrow$ 变成 42.5°C
- ...

关键性质：每次操作后，误差至少减半！

这就是”收缩映射”：每次操作让你离目标更近，而且有**保证的进步**。

值迭代就是这样：每次 Bellman 更新，误差至少乘以 γ (比如 0.9)。

4.1 收缩映射与 Banach 不动点定理

定义 4.1 (收缩映射). 算子 T 是 γ -收缩映射，如果对所有 V, V' ：

$$\|TV - TV'\|_{\infty} \leq \gamma \|V - V'\|_{\infty}$$

例 4.2 (理解收缩映射——橡皮筋). 想象一根橡皮筋上有两个点 V 和 V' ，距离是 d 。

”收缩映射”就像**同时拉两个点**，但拉完之后距离变成 $\gamma \cdot d$ 。

如果 $\gamma = 0.9$ ，每次操作距离缩短 10%。如果 $\gamma = 0.5$ ，每次操作距离缩短 50%。

不管一开始两点离多远，反复操作后它们会**越来越近**，最终重合！

重合的那个点就是**不动点**——所有起点最终都会到达那里。

定理 4.1 (Bellman 算子是收缩映射). Bellman 最优算子 $(T^*V)(s) = \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a)V(s')]$ 是 γ -收缩。

证明. 对任意 V, V' 和状态 s ，设 $a^* = \arg \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a)V(s')]$ 。

$$\begin{aligned} (T^*V)(s) - (T^*V')(s) &\leq [R(s, a^*) + \gamma \sum_{s'} P(s'|s, a^*)V(s')] - [R(s, a^*) + \gamma \sum_{s'} P(s'|s, a^*)V'(s')] \\ &= \gamma \sum_{s'} P(s'|s, a^*)[V(s') - V'(s')] \\ &\leq \gamma \|V - V'\|_{\infty} \end{aligned}$$

类似地可证 $(T^*V')(s) - (T^*V)(s) \leq \gamma \|V - V'\|_\infty$ 。

取 $\max$ 得 $\|T^*V - T^*V'\|_\infty \leq \gamma \|V - V'\|_\infty$ 。

□

证明的直觉

为什么 Bellman 算子是收缩的？

因为 Bellman 更新里有个 γ 在前面！

$V' = R + \gamma \times (\text{未来价值})$

未来价值的误差被 γ 打了折扣，所以误差在传播过程中自然衰减。

定理 4.2 (Banach 不动点定理). 若 T 是完备度量空间上的 γ -收缩映射 ($\gamma < 1$)，则：

1. T 有**唯一**不动点 V^*
2. 从任意 V_0 出发， $V_k = T^k V_0 \rightarrow V^*$
3. 收敛速度： $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$

例 4.3 (Banach 定理的直觉——GPS 导航). 你要去一个目的地 (不动点 V^*)，但你不知道在哪。

GPS 的承诺： 每走一步，你离目的地的距离至少缩短 10%。

那么：

- 不管你从哪出发，最终一定能到
- 只有一个目的地 (唯一性)
- 走 k 步后，距离最多是初始的 0.9^k

值迭代就是这样的 GPS：每次 Bellman 更新保证让你更接近 V^* 。

Banach 不动点定理的证明. **存在性：** 取任意 V_0 ，令 $V_k = T^k V_0$ 。对 $m > n$ ：

$$\|V_m - V_n\| \leq \sum_{i=n}^{m-1} \|V_{i+1} - V_i\| \leq \sum_{i=n}^{m-1} \gamma^i \|V_1 - V_0\| \leq \frac{\gamma^n}{1-\gamma} \|V_1 - V_0\|$$

当 $n \rightarrow \infty$ ，右边趋于 0，所以 $\{V_k\}$ 是 Cauchy 序列，收敛到某个 V^* 。

由 T 的连续性， $TV^* = T(\lim_k V_k) = \lim_k TV_k = \lim_k V_{k+1} = V^*$ 。

唯一性： 若 $V^*, \tilde{V}^*$ 都是不动点，则：

$$\|V^* - \tilde{V}^*\| = \|TV^* - T\tilde{V}^*\| \leq \gamma \|V^* - \tilde{V}^*\|$$

只能 $V^* = \tilde{V}^*$ 。

□

Algorithm 1 值迭代 (Value Iteration)

```
1: 初始化  $V_0(s) = 0$  对所有  $s$ 
2: for  $k = 0, 1, 2, \dots$  do
3:   for 每个状态  $s$  do
4:      $V_{k+1}(s) = \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s')]$ 
5:   end for
6:   if  $\|V_{k+1} - V_k\|_\infty < \epsilon(1 - \gamma)/(2\gamma)$  then
7:     停止
8:   end if
9: end for
10: 输出  $\pi(s) = \arg \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s')]$ 
```

收敛速度

$\gamma = 0.9$, 初始误差 $\|V_0 - V^*\| = 100$ 。

迭代次数 k	γ^k	误差上界
10	0.35	35
50	0.005	0.5
100	3×10^{-5}	0.003

误差每 $\log_{1/\gamma} 10 = \frac{\ln 10}{\ln(1/0.9)} \approx 22$ 次迭代减少 10 倍。

γ 越大, 收敛越慢: $\gamma = 0.99$ 需要 230 次迭代误差减 10 倍。

5 策略迭代: 更快的收敛

大白话

值迭代每次只更新一点点。策略迭代更激进:

1. **策略评估:** 完全解出当前策略 π 的值函数 V^π
2. **策略改进:** 根据 V^π 贪婪地更新策略

每次跳到”当前策略下的最优”, 而不是一点点挪。

定理 5.1 (策略改进定理). 设 $\pi'(s) = \arg \max_a Q^\pi(s, a)$ 。则 $V^{\pi'}(s) \geq V^\pi(s)$ 对所有 s 成立。
等号成立当且仅当 π 已经是最优策略。

证明. 对任意 s :

$$\begin{aligned} V^\pi(s) &\leq \max_a Q^\pi(s, a) = Q^\pi(s, \pi'(s)) \\ &= R(s, \pi'(s)) + \gamma \sum_{s'} P(s'|s, \pi'(s)) V^\pi(s') \\ &\leq R(s, \pi'(s)) + \gamma \sum_{s'} P(s'|s, \pi'(s)) \max_{a'} Q^\pi(s', a') \end{aligned}$$

继续展开, 得到 $V^\pi(s) \leq V^{\pi'}(s)$ 。

□

定理 5.2 (策略迭代有限步收敛). 策略迭代最多 $|A|^{|S|}$ 次迭代收敛到最优策略。

实际中通常只需 5-20 次迭代。

6 Monte Carlo 方法: 最朴素的 model-free 方法

大白话

值迭代、策略迭代都假设我们完全知道 P, R 。

现实里更常见的场景: 你有一个能跑的环境 (机器人、游戏、模拟器), 可以执行动作、观察反馈, 但 $P(s'|s, a)$ 和 $R(s, a)$ 的解析形式不知道。

最朴素的对策: 跑很多次, 对每个状态出现时的累积回报取样本均值——这就是 Monte Carlo 方法。

它是 bandit 最直接的 MDP 版本: 把”拉一次得到 r ”换成”跑完一个 episode 得到 G_t ”, 剩下全靠大数定律。

6.1 Bandit 的直接推广

把状态当成” 轨迹老虎机”

上周 bandit 的核心套路：

- 不知道 μ_a
- 拉臂 a 多次，记录奖励 r
- 用样本均值 $\hat{\mu}_a = \frac{1}{N_a} \sum_{i=1}^{N_a} r_i$ 估计 μ_a
- 大数定律 $\Rightarrow \hat{\mu}_a \rightarrow \mu_a$

MDP 里把” 臂” 换成” 状态”， ” 一次奖励” 换成” 从该状态出发的整段回报 G_t ”：

- 不知道 $V^\pi(s)$
- 跑 π 多次，每次记录从 s 出发观察到的 G_t
- 用样本均值 $\hat{V}(s) = \frac{1}{N_s} \sum_i G^{(i)}$ 估计 $V^\pi(s)$

大数定律仍然保证收敛——因为定义上 $V^\pi(s) = \mathbb{E}[G_t | s_t = s]$ ，样本均值估期望嘛。

这就是 **Monte Carlo** 方法。它和 bandit 是同一个统计原理，只不过把” 瞬时奖励” 换成了” 未来累积折扣奖励”。

6.2 MC 评估算法 (prediction)

定义 6.1 (首次访问 MC 评估). 给定策略 π 。运行 K 条完整轨迹：

Algorithm 2 First-Visit Monte Carlo 评估

- 1: 初始化 $V(s) = 0$ ，访问计数 $N(s) = 0$
 - 2: **for** episode $k = 1, \dots, K$ **do**
 - 3: 用 π 生成完整轨迹 $(s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T)$
 - 4: 计算 $G_t = \sum_{j=t}^{T-1} \gamma^{j-t} r_j$ 对每个 t
 - 5: **for** 每个在该轨迹中首次出现的状态 s ，对应时刻 t **do**
 - 6: $N(s) += 1$
 - 7: $V(s) \leftarrow V(s) + \frac{1}{N(s)} [G_t - V(s)]$
 - 8: **end for**
 - 9: **end for**
-

更新规则展开等价于增量样本均值：

$$V_n(s) = \frac{1}{n} \sum_{i=1}^n G^{(i)}$$

注记 6.1 (首次访问 vs 每次访问). **First-visit MC**: 一条轨迹里 s 出现多次只用第一次的 G 。样本独立，方差分析干净。

Every-visit MC: s 每次出现都更新。样本量更大但相关。

两者都 w.p.1 收敛到 V^π 。第一种严格无偏，第二种渐近无偏。实践用 every-visit 的多（数据利用率高）。

6.3 MC 的统计性质

定理 6.1 (MC 评估的收敛). 若每个状态 s 被无限次访问，且步长满足 $\sum_n \alpha_n = \infty, \sum_n \alpha_n^2 < \infty$ (或用 $\alpha_n = 1/n$)，则

$$V_n(s) \xrightarrow{w.p.1} V^\pi(s)$$

证明思路

这不是新结果。 $V^\pi(s) = \mathbb{E}[G_t | s_t = s]$ 是定义， $\{G^{(i)}\}$ （首次访问版本）是 i.i.d. 样本，强大数定律即得。

MC 评估在统计学上就是 bandit 的样本均值定理在 MDP 上的复读机版本。没有新的概率工具，只是把“奖励”重新定义成“未来累积折扣奖励”。

MC 的方差量级

单次 G_t 的方差，假设每步奖励方差 σ^2 、各步独立（最乐观情形）：

$$\text{Var}(G_t) = \sum_{k=0}^{\infty} \gamma^{2k} \sigma^2 = \frac{\sigma^2}{1 - \gamma^2}$$

$\gamma = 0.9 \Rightarrow$ 放大 ≈ 5.3 倍； $\gamma = 0.99 \Rightarrow$ 放大 ≈ 50 倍。

样本均值的标准差是 $\sqrt{\text{Var}(G_t)/n}$ ，要达到精度 ϵ 需要：

$$n = O\left(\frac{\sigma^2}{(1 - \gamma^2)\epsilon^2}\right)$$

轨迹。 γ 越接近 1，常数爆炸。

实际情形（状态转移随机性叠加奖励随机性）方差更大。这是 MC 的根本痛点。

6.4 MC control: 评估 + 改进的循环

Prediction 解决了，control（找最优 π ）怎么办？沿用策略迭代的 GPI 框架：评估 + greedy 改进。

Model-free control 的关键切换：必须学 Q 不学 V

策略迭代的改进步骤：

$$\pi'(s) = \arg \max_a Q^\pi(s, a)$$

如果只有 V^π ，model-based 下可以算 Q ：

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s')$$

——这需要 P, R ！

所以：**model-free control** 必须直接估计 Q ，不能只估 V 。这是从 prediction（学 V 就够）跳到 control（必须学 Q ）的根本切换点。

记住这个原则——下一章的 TD 也会撞上同一堵墙，得出同样的结论。

Algorithm 3 MC Control (ϵ -greedy 版)

- 1: 初始化 $Q(s, a)$ 任意， $\pi = \epsilon$ -greedy w.r.t. Q
 - 2: **for** episode $k = 1, 2, \dots$ **do**
 - 3: 用 π 生成完整轨迹
 - 4: 对每个访问到的 (s, a) : $Q(s, a) \leftarrow Q(s, a) + \alpha[G - Q(s, a)]$
 - 5: $\pi \leftarrow \epsilon$ -greedy w.r.t. 更新后的 Q
 - 6: **end for**
-

注记 6.2 (为什么需要 ϵ -greedy). 纯 greedy 会漏掉好动作——没访问过的 (s, a) 永远没有 Q 估计、永远不会被选中、永远没机会被试。 ϵ -greedy 保证每个动作都有 $\geq \epsilon/|A|$ 的概率被尝试，对应 bandit 的 **forced exploration**。

GLIE 条件 ($\epsilon \rightarrow 0$ 、每个 (s, a) 被无限次访问) 下，MC control w.p.1 收敛到 Q^* 。

6.5 MC 的两个软肋

MC 虽然简单优雅、和 bandit 理论同源，但有两个严重缺陷：

MC 不适用的场景

1. 必须等到 **episode** 结束才能算 G_t 。
 - 无限 horizon 任务（连续运行的工厂控制、未通关的游戏）根本没法用
 - 即使有限 horizon，episode 长意味着延迟更新——在线学习效率低
2. 方差极大：
 - $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$ 是 $O(1/(1-\gamma))$ 步噪声的叠加
 - Bandit 拉一次承担一步随机性，MC 拉一次承担一整段
 - 学习率很难调——大了不稳，小了慢

两条改进路线 = 后面两章

MC 的两个软肋分别催生了后面两章的主题。

软肋 1（样本效率低、方差大） $\Rightarrow$ §UCB-VI: 与其暴力 sample-average 整段回报，不如从样本里估出模型 $\hat{P}, \hat{R}$ ，再做 optimistic planning。把所有样本的信息塞进模型，对每个 (s, a) 都做了“信息共享”——这是 **model-based RL**，能拿到 $\tilde{O}(\sqrt{H^3 SAK})$ 的 regret。

软肋 2（必须等 episode 结束） $\Rightarrow$ §时序差分学习: 用 **bootstrap**——一步样本 + 自己当前的估计——绕过等待，每步都能更新。保持 model-free 和 online，导出 SARSA 和 Q-learning。

后续这两章请始终回头比较：

- UCB-VI: sample $\rightarrow$ 估模型 $\rightarrow$ 计划（每步 $O(SA \cdot S)$ 存储 + planning 开销）
- TD: sample $\rightarrow$ 一步 bootstrap $\rightarrow$ 更新 Q （每步 $O(1)$ ）
- MC: sample $\rightarrow$ 等到底 $\rightarrow$ 取均值（每步 $O(1)$ 但要等 episode）

三者各自付出不同代价，换不同的好处。

7 从 Bandit Regret 到 RL Regret

承接上周

上周定义的 Bandit Regret: $\sum_{t=1}^T (\mu^* - \mu_{a_t})$

MDP 的 Regret 类似，但要考虑 **episode 结构**：

定义 7.1 (Episodic MDP 的 Regret). 运行 K 个 episode，每个 episode 长度 H ：

$$\text{Regret}(K) = \sum_{k=1}^K \left[V_1^*(s_1^k) - V_1^{\pi_k}(s_1^k) \right]$$

其中 π_k 是第 k 个 episode 使用的策略。

Bandit vs MDP Regret

	Bandit	MDP
单轮损失	$\mu^* - \mu_a$	$V^*(s) - V^\pi(s)$
累积	T 轮	K 个 episode
最优界	$O(\sqrt{KT})$	$O(\sqrt{H^3 SAK})$

MDP 多了 $\sqrt{H^3 SA}$ 因子：状态空间、动作空间、时间范围都增加了探索难度。

8 UCB-VI: 把 UCB 推广到 MDP

相对 MC 的改进点

MC 暴力 sample-average 整段回报，样本效率低。UCB-VI 走另一条路：从样本里估出模型 $\hat{P}, \hat{R}$ ，再做 optimistic planning。

- MC: 每个 (s, a) 的回报独立估计，信息不共享
- UCB-VI: 所有样本都塞进模型 $\hat{P}, \hat{R}$ 里，不同 (s, a) 通过 planning 共享信息

代价：要存 $O(S^2 A)$ 的转移计数表 + 每个 episode 跑一次值迭代。回报： $\tilde{O}(\sqrt{H^3 S A K})$ regret，远优于 MC 的样本复杂度。

下面具体看怎么做。

核心思想

上周的 UCB: $\hat{\mu}_a + \sqrt{\frac{2 \ln t}{n_a}}$ (均值 + 不确定性 bonus)

UCB-VI 的思想完全一样：对不确定的转移概率，假设它们对我们有利。

具体做法：在值迭代中，给 Q 值加一个 bonus，bonus 大小与 (s, a) 的访问次数成反比。

例 8.1 (UCB-VI 的生活版——探索新城市)。你刚搬到一个新城市，想找最好的生活方式。

状态：你在哪个街区

动作：去哪个餐厅/商店/公园

问题：你不知道每个地方有多好，也不知道去了之后会遇到什么（转移概率）。

UCB-VI 策略：

1. 对每个没去过的地方，乐观假设它很棒（给高分）
2. 去过几次的地方，根据经验评分，但加一个“不确定性 bonus”
3. 去过很多次的地方，bonus 很小，评分接近真实

效果：

- 一开始疯狂探索（所有地方 bonus 都高）
- 慢慢 bonus 降低，开始 exploitation
- 最终找到最好的生活方式

和上周 UCB 的区别：

- UCB: 只考虑“这个餐厅有多好”
- UCB-VI: 还要考虑“去完这个餐厅，我会到哪个状态，那个状态有多好”

Algorithm 4 UCB-VI (完整版)

```
1: 初始化计数  $N(s, a) = 0$ ,  $N(s, a, s') = 0$ 
2: for episode  $k = 1, 2, \dots, K$  do
3:   // 乐观值迭代 (从后往前)
4:    $V_{H+1}(s) = 0$  for all  $s$ 
5:   for  $h = H, H-1, \dots, 1$  do
6:     for each  $(s, a)$  do
7:       if  $N(s, a) > 0$  then
8:          $\hat{P}(s'|s, a) = N(s, a, s')/N(s, a)$ 
9:          $\text{bonus}_h(s, a) = c\sqrt{\frac{H^2\iota}{N(s, a)}}$  where  $\iota = \ln(SAHK/\delta)$ 
10:      else
11:         $\text{bonus}_h(s, a) = H$  ▷ 最大可能值
12:      end if
13:       $Q_h(s, a) = \min \left\{ H, R(s, a) + \sum_{s'} \hat{P}(s'|s, a) V_{h+1}(s') + \text{bonus}_h(s, a) \right\}$ 
14:    end for
15:     $V_h(s) = \max_a Q_h(s, a)$ ,  $\pi_h^k(s) = \arg \max_a Q_h(s, a)$ 
16:  end for
17:  // 执行策略并收集数据
18:  从  $s_1^k$  出发, 执行  $\pi^k$ , 观察轨迹  $(s_1, a_1, r_1, \dots, s_H, a_H, r_H)$ 
19:  更新计数:  $N(s_h, a_h) += 1$ ,  $N(s_h, a_h, s_{h+1}) += 1$ 
20: end for
```

8.1 UCB-VI 的 Regret 分析

定理 8.1 (UCB-VI 的 Regret 界). 以概率至少 $1 - \delta$, 有

$$\text{Regret}(K) = \sum_{k=1}^K \left[V_1^*(s_1^k) - V_1^{\pi^k}(s_1^k) \right] \leq O\left(\sqrt{H^3 SAK\iota}\right),$$

其中

$$\iota = \ln(SAHK/\delta).$$

证明. 证明分四步。

Step 1: 建立好事件

定义好事件 $\mathcal{E}$: 对所有 (s, a, h, k) , 经验转移概率的误差均被 bonus 控制, 即

$$\left| \sum_{s'} \left(\hat{P}^k(s'|s, a) - P(s'|s, a) \right) V_{h+1}^k(s') \right| \leq \text{bonus}_h^k(s, a),$$

其中

$$\text{bonus}_h^k(s, a) = c\sqrt{\frac{H^2\iota}{N^k(s, a) \vee 1}}.$$

这里 $N^k(s, a)$ 表示在 episode k 开始前, 状态动作对 (s, a) 被访问的总次数。注意这里的计数不依赖于 h 。

引理 8.1 (好事件成立). 选择足够大的常数 c , 有

$$\mathbb{P}(\mathcal{E}) \geq 1 - \delta.$$

引理证明. 固定 (s, a, h, k) . 设在 episode k 开始前, (s, a) 已经被访问了

$$n = N^k(s, a)$$

次。由于 $V_{h+1}^k(s') \in [0, H]$, 由 Hoeffding 不等式,

$$\mathbb{P}\left(\left|\sum_{s'}(\hat{P}^k - P)(s'|s, a)V_{h+1}^k(s')\right| > \epsilon\right) \leq 2 \exp\left(-\frac{2n\epsilon^2}{H^2}\right).$$

取

$$\epsilon = c\sqrt{\frac{H^2\iota}{n \vee 1}},$$

则失败概率至多为

$$2 \exp(-2c^2\iota) \leq \frac{\delta}{SAHK},$$

其中最后一步可通过选择足够大的常数 c 保证。

对所有 (s, a, h, k) 作 union bound, 得到

$$\mathbb{P}(\mathcal{E}) \geq 1 - \delta.$$

□

Step 2: 乐观性

引理 8.2 (乐观引理). 在好事件 $\mathcal{E}$ 下, 对所有 (s, h, k) , 有

$$V_h^k(s) \geq V_h^*(s).$$

引理证明. 对 h 从 $H+1$ 到 1 作反向归纳。

当 $h = H+1$ 时,

$$V_{H+1}^k(s) = 0 = V_{H+1}^*(s).$$

假设对 $h+1$ 有

$$V_{h+1}^k(s) \geq V_{h+1}^*(s).$$

则对任意 (s, a) ,

$$\begin{aligned} Q_h^k(s, a) &= R(s, a) + \sum_{s'} \hat{P}^k(s'|s, a)V_{h+1}^k(s') + \text{bonus}_h^k(s, a) \\ &\geq R(s, a) + \sum_{s'} P(s'|s, a)V_{h+1}^k(s') \\ &\geq R(s, a) + \sum_{s'} P(s'|s, a)V_{h+1}^*(s') \\ &= Q_h^*(s, a). \end{aligned}$$

第一处不等式使用了好事件, 第二处不等式使用了归纳假设。

因此

$$V_h^k(s) = \max_a Q_h^k(s, a) \geq \max_a Q_h^*(s, a) = V_h^*(s).$$

归纳完成。 □

Step 3: Regret 分解

由乐观性,

$$\begin{aligned} \text{Regret}(K) &= \sum_{k=1}^K \left[V_1^*(s_1^k) - V_1^{\pi^k}(s_1^k) \right] \\ &\leq \sum_{k=1}^K \left[V_1^k(s_1^k) - V_1^{\pi^k}(s_1^k) \right]. \end{aligned}$$

下面分析单个 episode 中

$$V_1^k(s_1^k) - V_1^{\pi^k}(s_1^k)$$

的大小。

引理 8.3 (单 episode 分解). 在好事件 $\mathcal{E}$ 下, 对每个 episode k , 有

$$V_1^k(s_1^k) - V_1^{\pi^k}(s_1^k) \leq \sum_{h=1}^H \left[2 \text{bonus}_h^k(s_h^k, a_h^k) + \xi_h^k \right],$$

其中 ξ_h^k 是鞅差序列。

引理证明. 定义

$$\delta_h^k(s) = V_h^k(s) - V_h^{\pi^k}(s).$$

在 episode k 的真实轨迹上, 记

$$\delta_h^k = \delta_h^k(s_h^k).$$

由于策略 π^k 在第 h 步选择

$$a_h^k = \arg \max_a Q_h^k(s_h^k, a),$$

有

$$\begin{aligned} \delta_h^k &= V_h^k(s_h^k) - V_h^{\pi^k}(s_h^k) \\ &= Q_h^k(s_h^k, a_h^k) - \left[R(s_h^k, a_h^k) + \sum_{s'} P(s'|s_h^k, a_h^k) V_{h+1}^{\pi^k}(s') \right] \\ &= \sum_{s'} \hat{P}^k(s'|s_h^k, a_h^k) V_{h+1}^k(s') - \sum_{s'} P(s'|s_h^k, a_h^k) V_{h+1}^{\pi^k}(s') + \text{bonus}_h^k(s_h^k, a_h^k). \end{aligned}$$

将第一项加減

$$\sum_{s'} P(s'|s_h^k, a_h^k) V_{h+1}^k(s')$$

得到

$$\begin{aligned}
\delta_h^k &= \sum_{s'} (\hat{P}^k - P)(s'|s_h^k, a_h^k) V_{h+1}^k(s') \\
&\quad + \sum_{s'} P(s'|s_h^k, a_h^k) \left[V_{h+1}^k(s') - V_{h+1}^{\pi^k}(s') \right] + \text{bonus}_h^k(s_h^k, a_h^k) \\
&\leq 2 \text{bonus}_h^k(s_h^k, a_h^k) + \mathbb{E} \left[\delta_{h+1}^k(s_{h+1}^k) \mid s_h^k, a_h^k \right],
\end{aligned}$$

其中最后一步使用了好事件。

定义鞅差

$$\xi_h^k = \delta_{h+1}^k(s_{h+1}^k) - \mathbb{E} \left[\delta_{h+1}^k(s_{h+1}^k) \mid s_h^k, a_h^k \right].$$

于是

$$\mathbb{E} \left[\delta_{h+1}^k(s_{h+1}^k) \mid s_h^k, a_h^k \right] = \delta_{h+1}^k(s_{h+1}^k) - \xi_h^k.$$

因此

$$\delta_h^k \leq 2 \text{bonus}_h^k(s_h^k, a_h^k) + \delta_{h+1}^k(s_{h+1}^k) - \xi_h^k.$$

将上式从 $h = 1$ 递推到 H ，并利用

$$\delta_{H+1}^k = 0,$$

得到

$$\delta_1^k \leq \sum_{h=1}^H \left[2 \text{bonus}_h^k(s_h^k, a_h^k) - \xi_h^k \right].$$

由于 $-\xi_h^k$ 仍然是鞅差项，为了记号简洁，重新记作 ξ_h^k ，得到

$$\delta_1^k \leq \sum_{h=1}^H \left[2 \text{bonus}_h^k(s_h^k, a_h^k) + \xi_h^k \right].$$

□

Step 4: 累积 bonus 的界

由 Step 3,

$$\text{Regret}(K) \leq 2 \sum_{k=1}^K \sum_{h=1}^H \text{bonus}_h^k(s_h^k, a_h^k) + \sum_{k=1}^K \sum_{h=1}^H \xi_h^k.$$

首先处理鞅差项。由于

$$|\xi_h^k| \leq H,$$

由 Azuma-Hoeffding 不等式，以高概率有

$$\left| \sum_{k=1}^K \sum_{h=1}^H \xi_h^k \right| \leq O \left(H \sqrt{HK\iota} \right).$$

下面处理 bonus 项。由定义，

$$\sum_{k=1}^K \sum_{h=1}^H \text{bonus}_h^k(s_h^k, a_h^k) = cH\sqrt{\iota} \sum_{k=1}^K \sum_{h=1}^H \frac{1}{\sqrt{N^k(s_h^k, a_h^k) \vee 1}}.$$

对每个固定的 (s, a) ，设它在所有 K 个 episode 中总共被访问了

$$n(s, a)$$

次，并把这些访问时刻依次记为

$$t_1, t_2, \dots, t_{n(s, a)}.$$

在第 i 次访问时，有

$$N^k(s, a) = i - 1.$$

因此

$$\sum_{i=1}^{n(s, a)} \frac{1}{\sqrt{(i-1) \vee 1}} \leq 1 + \sum_{i=1}^{n(s, a)-1} \frac{1}{\sqrt{i}} \leq 2\sqrt{n(s, a)}.$$

于是

$$\begin{aligned} \sum_{k=1}^K \sum_{h=1}^H \frac{1}{\sqrt{N^k(s_h^k, a_h^k) \vee 1}} &\leq \sum_{s, a} 2\sqrt{n(s, a)} \\ &\leq 2\sqrt{SA} \sqrt{\sum_{s, a} n(s, a)} \\ &= 2\sqrt{SA} \sqrt{HK} \\ &= 2\sqrt{SAHK}. \end{aligned}$$

其中第二步使用 Cauchy-Schwarz，不再对 h 分桶；并且

$$\sum_{s, a} n(s, a) = HK.$$

因此

$$\sum_{k=1}^K \sum_{h=1}^H \text{bonus}_h^k(s_h^k, a_h^k) \leq O\left(H\sqrt{SAHK\iota}\right) = O\left(\sqrt{H^3SAK\iota}\right).$$

综上，

$$\text{Regret}(K) \leq O\left(\sqrt{H^3SAK\iota}\right) + O\left(H\sqrt{HK\iota}\right).$$

当 $SA \geq 1$ 时，第一项主导，因此

$$\text{Regret}(K) \leq O\left(\sqrt{H^3SAK\iota}\right).$$

证明完成。 □

UCB-VI Regret 数值验证

$|S| = 100$, $|A| = 4$, $H = 20$, $K = 10000$, $\delta = 0.05$:

$\iota = \ln(100 \times 4 \times 20 \times 10000/0.05) = \ln(1.6 \times 10^9) \approx 21$

$\text{Regret} \leq c\sqrt{20^3 \times 100 \times 4 \times 10000 \times 21}$

$= c\sqrt{8000 \times 400 \times 10000 \times 21} = c\sqrt{6.72 \times 10^{11}} \approx 820000 \cdot c$

取 $c = 1$ (实际常数更小)，平均每 episode $\text{Regret} \approx 82$ 。

实际中 Regret 通常比理论界小 5-10 倍。

8.2 Regret 下界

定理 8.2 (信息论下界). 存在 MDP 使得任何算法的期望 Regret 至少:

$$\mathbb{E}[\text{Regret}(K)] \geq \Omega(\sqrt{HSAK})$$

证明. 构造困难实例:

考虑”链式 MDP”:

- S 个状态排成一条链: $s_1 \rightarrow s_2 \rightarrow \cdots \rightarrow s_S$
- 每个状态有 A 个动作
- 只有一个”正确”动作能前进, 其他动作原地不动
- 正确动作是随机选的 (算法不知道)
- 到达 s_S 获得奖励 1, 否则为 0

分析:

要到达 s_S , 必须在每个状态 s_i 找到正确动作。

在状态 s_i , 需要 $\Omega(A)$ 次尝试才能找到正确动作 (每个动作概率 $1/A$)。

总共需要 $\Omega(SA)$ 次探索才能学会完整策略。

在探索期间, 每个 episode 的期望 Regret 是 $\Omega(1)$ (因为很可能到不了 s_S)。

设探索需要 T 个 episode, 则 $\text{Regret} \geq \Omega(T)$ 。

但我们也需要 $T \cdot H$ 次转移来完成探索。每次转移最多给一个 (s, a) 提供信息。

要区分每个 (s, a) 是否”正确”, 由上周的信息论下界, 需要 $\Omega(1)$ 次访问。

总共需要 $\Omega(SA)$ 次访问, 即 $T \cdot H \geq \Omega(SA)$, 所以 $T \geq \Omega(SA/H)$ 。

但这还不够紧。更精细的分析:

关键洞察: 在 K 个 episode 中, 每个 (s, a, h) 的访问次数约为 $K/(SA)$ (平均分配)。

用上周 Lai-Robbins 风格的论证: 要区分两个相差 Δ 的分布, 需要 $\Omega(1/\Delta^2)$ 样本。

对手可以选择 $\Delta = \Theta(1/\sqrt{K/(SA)}) = \Theta(\sqrt{SA/K})$ 。

此时单个状态的 Regret 贡献 $\approx \Delta \cdot K/(SA) \cdot (SA) = \Delta K = \Theta(\sqrt{SAK})$ 。

乘以 H 步的误差传播: $\text{Regret} \geq \Omega(\sqrt{HSAK})$ 。 □

上界与下界的 gap

界	Regret
UCB-VI 上界	$\tilde{O}(\sqrt{H^3SAK})$
下界	$\Omega(\sqrt{HSAK})$
Gap	H 的指数: 1.5 vs 0.5

Gap 在 H 上。近年有改进算法达到 $\tilde{O}(\sqrt{H^2SAK})$, 但 $\sqrt{H}$ 的 gap 是否能消除仍是开放问题。

9 时序差分学习：从 prediction 到 control

大白话

回顾迄今为止处理 MDP 的三条路：

- **DP**（值迭代/策略迭代）：要 P, R 的解析形式，每步算期望
- **Monte Carlo**：model-free，但要等整段 **episode** 才能更新，方差大
- **UCB-VI**：从样本估出 $\hat{P}, \hat{R}$ 再 planning（model-based，乐观探索）

本节给出第四条路：**时序差分（Temporal Difference, TD）学习**。它是 model-free（像 MC）但用 **bootstrap** 绕过等待（像 DP）：用一步样本 + 当前估计代替整段 G_t 。

TD = bandit 式样本 + DP 式 bootstrap，恰好填上 MC 和 DP 之间的空位。

Q-learning 和 SARSA 都不是凭空出现的算法——它们是 TD 框架在 control 问题上的两个具体实例，差别只在 target 里 a' 怎么选（按 π vs. 取 $\max$ ）。

9.1 Prediction 问题：给定策略，估计价值

问题设定

固定一个策略 π 。我们不知道 P, R ，但可以让 π 在环境中跑，观察序列：

$$s_0 \xrightarrow{a_0 \sim \pi(\cdot | s_0)} r_0, s_1 \xrightarrow{a_1 \sim \pi(\cdot | s_1)} r_1, s_2 \rightarrow \dots$$

目标：估计 $V^\pi(s)$ 。

注意这不是求最优策略——是给定 π 估它的值。术语上区分：

- **Prediction**（评估）：给 π 估 V^π 或 Q^π
- **Control**（控制）：找最优 π^*

本节先讲 prediction（更简单、更基础），再过渡到 control。

估计 V^π 的三条路

回忆 Bellman 期望方程：

$$V^\pi(s) = \mathbb{E}_{a \sim \pi} \left[R(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} [V^\pi(s')] \right]$$

右边有两层期望（动作 + 下一状态），还有 V^π 自身的递归。处理这三件事的方式不同，得到三种算法：

方法	要 P, R ?	用样本?	怎么处理 V^π 的递归
DP（策略评估）	是	否	用前一轮估计 V_k （bootstrap）
Monte Carlo	否	是（整段）	不递归 ，直接用真实 G_t
TD(0)	否	是（一步）	用当前估计 $V(s')$ （bootstrap）

两个轴：是否用样本（vs. 解析期望）、是否 bootstrap（vs. 用真实回报）。DP 和 TD 都 bootstrap，DP 用期望、TD 用样本；MC 不 bootstrap 但用样本。**TD = bandit 式样本 + DP 式 bootstrap。**

9.2 TD(0)：用 bootstrap 替换 G_t

回顾 MC 的痛点

上一章 MC 的核心痛点是**要等**：从 s_t 出发，必须把整段轨迹跑完才能算 G_t ，才能更新 $V(s_t)$ 。

TD 的诞生动机就是：能不能不等？**一步就更新**？

TD 的核心 idea 极其简单：

Return 的递归关系

$$G_t = r_t + \gamma G_{t+1}$$

如果手头已经有 $V(s_{t+1})$ 作为 $\mathbb{E}[G_{t+1} | s_{t+1}]$ 的估计，就可以用它代替未知的 G_{t+1} ：

$$G_t \approx r_t + \gamma V(s_{t+1})$$

一步就能更新，不用等到 episode 结束。这个“用自己的估计替换真值”的动作，就叫 **bootstrap**。

定义 9.1 (TD(0) 算法——prediction 版). 观察到 (s_t, a_t, r_t, s_{t+1}) 后：

$$V(s_t) \leftarrow V(s_t) + \alpha \left[\underbrace{r_t + \gamma V(s_{t+1})}_{\text{TD target}} - V(s_t) \right]$$

定义 **TD 误差**： $\delta_t := r_t + \gamma V(s_{t+1}) - V(s_t)$ 。

更新简写为 $V(s_t) \leftarrow V(s_t) + \alpha \delta_t$ 。

Bootstrapping 的正式定义

一个学习算法 **bootstrap**，当且仅当：它的更新目标含当前估计的价值函数 V 或 Q 。

- DP（策略评估/值迭代）bootstrap: $V_{k+1}(s) = \mathbb{E}[r + \gamma V_k(s')]$, target 里有 V_k
- TD(0) bootstrap: target = $r + \gamma V(s')$, 含 V
- MC 不 bootstrap: target = G_t , 全是真实奖励, 不含 V

Bootstrap 是一把双刃剑:

- 好处：一步即可更新（不等 episode）；方差降低（不叠加整段噪声）
- 代价：引入偏差（ V 估计有错, target 也错）；和函数逼近 + **off-policy** 组合时会发散 (§Deadly Triad)

记住这个定义——后面讨论 Deadly Triad、advantage estimation、GAE, "bootstrap" 都是核心词。

例 9.1 (MC vs TD: 一个具体轨迹). 轨迹: $s_0 \rightarrow (r_0 = 1) \rightarrow s_1 \rightarrow (r_1 = 2) \rightarrow s_2 \rightarrow (r_2 = 4) \rightarrow$ 终止。 $\gamma = 1$ 。

当前估计: $V(s_0) = 10, V(s_1) = 5, V(s_2) = 3$ 。 $\alpha = 0.1$ 。

MC 对 s_0 的更新:

- $G_0 = 1 + 2 + 4 = 7$ （真实回报）
- Target = 7, 误差 = $7 - 10 = -3$
- $V(s_0) \leftarrow 10 + 0.1 \times (-3) = 9.7$

TD(0) 对 s_0 的更新（在看到 s_1 后立刻发生，不等 episode 结束）:

- Target = $r_0 + \gamma V(s_1) = 1 + 5 = 6$
- TD 误差 $\delta_0 = 6 - 10 = -4$
- $V(s_0) \leftarrow 10 + 0.1 \times (-4) = 9.6$

两个 target 不同: MC 用 7（真实但带 r_1, r_2 的方差），TD 用 6（用 $V(s_1) = 5$ 代替了未来——如果这个 5 是准的、target 偏差就小，但只承担一步的奖励噪声）。

为什么 TD 降方差

$$\text{Var}(\text{MC target}) = \text{Var}\left(\sum_{k=0}^{\infty} \gamma^k r_{t+k}\right)$$

$$\text{Var}(\text{TD target}) = \text{Var}(r_t + \gamma V(s_{t+1}))$$

只要 V 是个相对稳定的估计, TD target 只承担一步奖励的随机性 + 一个不太抖的 V 值, 方差远小于 MC target 那个无穷和。

代价: 若 $V(s_{t+1})$ 有偏差 b , TD target 有偏差 γb 。Bias-variance tradeoff——TD(λ) 正是要在这个权衡上插值 (见后面 §TD(λ))。

定理 9.1 (TD(0) 收敛性 - Sutton 1988; Dayan 1992). 表格情形下, 若:

1. 所有状态 s 被无限次访问
2. 学习率满足 $\sum_t \alpha_t(s) = \infty$, $\sum_t \alpha_t(s)^2 < \infty$

则 TD(0) 以概率 1 收敛到 V^π 。

证明思路

TD(0) 期望更新方向是

$$\mathbb{E}[\delta_t | s_t = s] = (T^\pi V)(s) - V(s)$$

其中 T^π 是 Bellman 期望算子——已在 §3 证明它是 γ -收缩的不动点 V^π 。

所以 TD(0) 是 $V_{k+1} = T^\pi V_k$ 这个 DP 迭代的随机逼近版本: 用样本 (s_t, r_t, s_{t+1}) 估计 $T^\pi V$ 。

收敛性证明结构和 Q-learning 相同 (§ 后面): 收缩算子 + 鞅差噪声 + Robbins-Monro 步长 $\rightarrow$ w.p.1 收敛。把 Q-learning 证明里的 T^* 换成 T^π 、 $\max_{a'}$ 换成 $\mathbb{E}_{a' \sim \pi}$ 即可。

三种 prediction 方法的统一视图

	Target	用模型	用样本
DP (策略评估)	$\sum_a \pi(a s)[R + \gamma \sum_{s'} P V_k(s')]$	是	否
MC 评估	G_t (完整回报)	否	是
TD(0)	$r_t + \gamma V(s_{t+1})$	否	是 (一步)

TD = DP 的递归 + MC 的样本。把 DP 里的”求期望”换成”采样”, 把 MC 里的”完整回报”换成”一步样本 + bootstrap”。

9.3 从 prediction 到 control: 为什么必须学 Q 不学 V

Prediction 解决了”给定 π 估值”。要找最优策略呢?

回忆策略迭代的 improvement 步骤:

$$\pi'(s) = \arg \max_a Q^\pi(s, a)$$

Model-free control 的根本约束

要在不知道 P, R 的情况下做策略改进，必须能比较不同动作的好坏：

$$\pi'(s) = \arg \max_a Q(s, a)$$

如果手头只有 V ，想算 Q 需要：

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s')$$

——这需要 P 和 R ！

所以：**model-free control 必须直接学 Q ，不学 V** 。这是从 prediction（学 V 就够）到 control（必须学 Q ）的根本驱动力。

把 TD(0) 从 V 推广到 Q 。需要的样本变为五元组 $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ （多了”下一步动作”）：

定义 9.2 (TD(0) for Q)。

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

这里 a_{t+1} 从哪来？这个看似平凡的选择，决定了你得到的是 on-policy 还是 off-policy 算法。两个不同的答案对应 SARSA 和 Q-learning。

9.4 SARSA: on-policy TD control

定义 9.3 (SARSA)。名字来自更新所用的五元组： $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ 。

Algorithm 5 SARSA

- 1: 初始化 $Q(s, a)$ 任意
 - 2: 在 s_0 用 ϵ -greedy（基于当前 Q ）选 a_0
 - 3: **for** 每步 t **do**
 - 4: 执行 a_t ，观察 r_t, s_{t+1}
 - 5: 在 s_{t+1} 用 ϵ -greedy 选 a_{t+1}
 - 6: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$
 - 7: $t \leftarrow t + 1$
 - 8: **end for**
-

关键： a_{t+1} 是即将真正执行的下一步动作，按当前 behavior policy（ ϵ -greedy）采样。

SARSA 是 on-policy 的

SARSA 的 TD target 里的 $a_{t+1} \sim \pi_{\text{behavior}}$ ——产生数据的策略和被评估的策略是同一个。

所以 SARSA 学到的 Q ，是当前 ϵ -greedy 策略本身的 Q^π （其中 $\pi = \epsilon$ -greedy w.r.t. Q ）。要让 $Q \rightarrow Q^*$ ，必须让 $\epsilon \rightarrow 0$ （**GLIE**: Greedy in the Limit with Infinite Exploration）：

- 长期 $\epsilon \rightarrow 0$ behavior policy $\rightarrow$ greedy
- 但 ϵ 下降不能太快，否则探索不充分（条件 1 失败）
- 常用： $\epsilon_t = 1/t$

9.5 Q-learning: off-policy TD control

现在做一个微小但关键的修改：把 SARSA target 里的 $Q(s_{t+1}, a_{t+1})$ 换成 $\max_{a'} Q(s_{t+1}, a')$ 。

定义 9.4 (Q-learning).

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

max 的语义：off-policy 的诞生

SARSA 的 target $Q(s_{t+1}, a_{t+1})$ 对应当前 ϵ -greedy 策略的下一步价值——评估的就是行为策略本身。

Q-learning 的 target $\max_{a'} Q(s_{t+1}, a')$ 对应一个不同的策略：

$$\pi_{\text{greedy}}(s) = \arg \max_a Q(s, a) \quad (\text{无探索的纯贪婪})$$

但智能体实际行动时仍用 ϵ -greedy（必须有探索才能更新所有 (s, a) ）。

这就是 **off-policy** 的根源：

- Behavior policy（产生数据）： ϵ -greedy
- Target policy（被学习/评估）：greedy
- 两者不同 off-policy

为什么这样做有意义：Q-learning 学到的是 Q^* （最优策略的 Q ），与你用什么探索策略无关（前提是所有 (s, a) 都能被访问到）。SARSA 要学 Q^* 得让 $\epsilon \rightarrow 0$ ；Q-learning 直接学 Q^* ， ϵ 只影响样本质量、不影响目标。这是 Q-learning 相对 SARSA 最大的优势。代价：函数逼近下会触发 Deadly Triad (§ 后面)。

SARSA vs Q-learning——同源异果

	SARSA	Q-learning
TD target	$r + \gamma Q(s', a')$	$r + \gamma \max_{a'} Q(s', a')$
a' 怎么选	从 π_{behavior} 采样	取 $\arg \max$
On/off-policy	on-policy	off-policy
评估的策略	behavior policy (ϵ -greedy)	greedy policy
学到什么	$Q^{\pi_{\text{behavior}}}$	Q^*
何时 $\rightarrow Q^*$	需要 GLIE ($\epsilon \rightarrow 0$)	直接收敛 (只要访问充分)
表格收敛性	w.p.1 (GLIE 下)	w.p.1 (Watkins 1992)

两个算法只差一个 $\max$ ，但语义完全不同——一个是“评估自己（含探索）”，一个是“评估假想的 greedy 自己”。

例 9.2 (悬崖行走：on-policy vs off-policy 的实际差别). 经典对比 (Sutton & Barto, Example 6.6): 智能体在网格世界，要从 S 走到 G。地图底部一长条是“悬崖”，掉下去 -100 奖励并送回起点；其他每步 -1 。

Q-learning 学到的策略：贴着悬崖边走最短路径（这是 π^* ）。但训练过程中智能体按 ϵ -greedy 行动，所以经常被 ϵ 的扰动推下悬崖，训练期累积奖励反而差。

SARSA 学到的策略：绕远路、远离悬崖。因为 SARSA 评估的就是 ϵ -greedy 策略本身——它“知道”自己以后还会随机出错，所以选一条容错率高的路。训练期累积奖励高，但学到的策略不是 π^* 。

道理：Q-learning 假装自己最终会完美，**SARSA** 承认自己永远会犯小错。两者各有适用场景：要部署时关探索，Q-learning 好；要训练中也少出大事故（机器人、医疗），SARSA 好。

Tabular RL 算法的二维分类表

现在我们终于可以把表格 RL 所有算法放在一张图里：

	Prediction (评估 π)	Control (找 π^*)
DP (要模型)	策略评估	值迭代 / 策略迭代
MC (无模型、不 bootstrap)	MC 评估	MC control + ϵ -greedy
TD on-policy	TD(0) on V	SARSA
TD off-policy	重要性采样 TD	Q-learning

记忆要点：

- 横轴：估值 vs 找最优
- 纵轴：怎么处理期望（解析 / 整段样本 / 一步样本 + bootstrap）和 on/off-policy
- Q-learning = TD + control + off-policy (max)
- SARSA = TD + control + on-policy (按 π 采样 a')

后面要讲的所有东西都长在这张表上：

- TD(λ)：在 TD(0)（一步）和 MC（整段）之间用 λ 插值
- Linear TD/Q：把表格换成 $\theta^\top \phi(s)$
- Deadly Triad：FA + bootstrap + off-policy 组合发散
- Advantage function / policy gradient (后续课)：TD 误差 δ_t 恰是 advantage $A^\pi(s_t, a_t)$ 的无偏估计
- Deep RL 的 DQN：Q-learning + 神经网络 + replay buffer + target network

笔记 9.1 (术语对照——避免课后混淆)。不同教材、论文对同一概念叫法不一，列在这里：

- **Prediction** = policy evaluation = 策略评估
- **Control** = optimization = 找最优策略
- **Bootstrap** = 用估计更新估计 (DP 和 TD 都 bootstrap, MC 不)
- **On-policy**: target policy = behavior policy
- **Off-policy**: target policy $\neq$ behavior policy
- **TD error** $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ (或 Q 版本)
- **TD target** $= r_t + \gamma V(s_{t+1})$ (TD 误差里 $V(s_t)$ 之外的部分)

10 核心概念与工程技巧：on/off-policy、经验回放、目标网络

大白话

到这里我们已经见过一个不小的算法动物园：值迭代、策略迭代、MC、UCB-VI、TD(0)、SARSA、Q-learning。它们看起来五花八门，但其实坐在同一个低维空间里——只要找对了坐标轴。

本节做两件事：

(1) 概念坐标。给出区分 RL 算法的三个最重要维度：

- **model-based vs model-free**（要不要 P, R ）——前几节的主线
- **on-policy vs off-policy**（执行的策略 = 评估的策略吗）
- **online vs offline**（边交互边学 vs 只用历史数据）

(2) 三件工程套件。介绍把上面算法做”实战”的三个核心技巧：

- 经验回放（**experience replay**）——让 online 算法重复消化样本
- 目标网络（**target network**）——让 bootstrap 稳定
- 重要性采样（**importance sampling**）——off-policy 的统计桥梁

外加一个常见陷阱：**Q-learning** 的过估计偏差与 **Double Q-learning**。
这些是后面看任何 deep RL 文献都绕不开的词汇。

10.1 On-policy vs off-policy：精确定义

定义 10.1 (Behavior policy 与 target policy). 任何 RL 算法在某个时刻都涉及两个策略：

- **Behavior policy π_b** : 产生数据的策略（”我实际怎么走、观察到了什么”）
- **Target policy π_t** : 被学习/评估的策略（”我想知道哪个策略的价值/我想优化哪个策略”）

On-policy: $\pi_b = \pi_t$ （评估和执行是同一个策略）

Off-policy: $\pi_b \neq \pi_t$ （用一个策略的数据学习另一个策略）

例 10.1 (把已学算法摆进去).

算法	π_b	π_t	是哪一种
MC 评估	π	π	on-policy
MC control	ϵ -greedy	ϵ -greedy	on-policy
SARSA	ϵ -greedy	ϵ -greedy	on-policy
Q-learning	ϵ -greedy	greedy	off-policy
UCB-VI	乐观 greedy	乐观 greedy	on-policy（执行 = 评估）

Q-learning 成为 off-policy 的唯一原因: target 里的 $\max_{a'} Q(s', a')$ 对应的是 greedy 策略 (π_t), 与实际行动的 ϵ -greedy (π_b) 不同。改成 target 里按 π_b 采样 a' , 立刻变回 on-policy

(即 SARSA)。

Off-policy 的好处

1. 探索与利用解耦：用充分探索的 π_b 产生数据，学到完全 greedy 的 π_t 。最优行为不被探索噪声污染。
2. 数据复用：旧数据（不管由哪个旧策略产生）可以反复利用——这是经验回放的前提
3. 演示/示教学习：可以从人类专家或别的策略的数据中学习
4. 多策略并行评估：用同一份数据同时评估多个候选 target policy

Off-policy 的代价

1. 分布不匹配： π_b 产生的状态/动作分布 $d^{\pi_b} \neq d^{\pi_t}$ ，朴素估计会有偏。修正工具：重要性采样（下面讲）
2. Bellman 最优算子的 max 带来过估计（见后 §Double Q-learning）
3. 和函数逼近 + bootstrap 组合 = Deadly Triad（见 §Linear FA）——deep RL 最常踩的坑

10.2 重要性采样：off-policy 的统计桥梁

大白话

要从 π_b 产生的样本估计 π_t 下的期望，最干净的统计工具是重要性采样（importance sampling, IS）。

直觉：用比率 $\rho = \pi_t/\pi_b$ 给每个样本加权——被 π_b 多观察到的事件权重降低，反之升高。

定理 10.1 (重要性采样恒等式)。

$$\mathbb{E}_{a \sim \pi_t}[f(a)] = \mathbb{E}_{a \sim \pi_b} \left[\frac{\pi_t(a)}{\pi_b(a)} f(a) \right] = \mathbb{E}_{a \sim \pi_b} [\rho(a) f(a)]$$

前提（覆盖性条件）： $\pi_t(a) > 0 \Rightarrow \pi_b(a) > 0$ 。

证明。

$$\mathbb{E}_{a \sim \pi_t}[f(a)] = \sum_a \pi_t(a) f(a) = \sum_a \pi_b(a) \cdot \frac{\pi_t(a)}{\pi_b(a)} f(a) = \mathbb{E}_{a \sim \pi_b} [\rho(a) f(a)]$$

两步代数。 □

多步 IS 与方差爆炸

对 T 步轨迹做 off-policy 评估，需要乘累积似然比：

$$\rho_{0:T-1} = \prod_{t=0}^{T-1} \frac{\pi_t(a_t|s_t)}{\pi_b(a_t|s_t)}$$

这个连乘让方差呈**指数爆炸**。若每步 $\rho_t \sim$ 均值 1、方差 σ^2 ，则 $\text{Var}(\rho_{0:T-1}) \sim (1+\sigma^2)^T - 1$ ——这就是为什么纯 IS 只在短 horizon 可行。

Q-learning 为什么不需要 IS

你也许会问：Q-learning 是 off-policy ($\pi_b = \epsilon$ -greedy, $\pi_t = \text{greedy}$)，为什么从来没出现似然比？

关键：Q-learning 的 target $r + \gamma \max_{a'} Q(s', a')$ 是 **Bellman optimality** 算子对 Q 的一步应用，只用到 (s, a, r, s') 这一步转移——和产生这个转移的策略无关（环境 dynamics P, R 不依赖策略）。

所以 Q-learning 评估的不是某个 target policy π_t 的”长期价值” Q^{π_t} ，而是**最优算子** T^* 的不动点 Q^* 。这绕开了”用 π_b 的轨迹估 π_t 的 return”那种需要 IS 校正的设定。

代价：换成了 max 带来的偏差——见下面的过估计。

这是 Q-learning 设计上的精妙之处：用**算子**（关于环境，与策略无关）代替**轨迹**（依赖策略），完美绕过 IS 的方差爆炸。

10.3 Online vs offline：另一个独立维度

定义 10.2 (Online vs offline). **Online：**算法在与环境的**持续交互**中学习。每步执行动作 $\rightarrow$ 看反馈 $\rightarrow$ 更新 $\rightarrow$ 继续执行。

Offline（也叫 batch RL）：只有一个预先收集好的数据集 $\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^N$ 。算法不能再与环境交互，全部学习在 $\mathcal{D}$ 上完成。

Off-policy 和 offline 是不同的轴——最容易混的两个概念

	Online	Offline
On-policy	SARSA, A2C, PPO	不存在（offline 下 π_b 固定无法 $= \pi_t$ ）
Off-policy	Q-learning, DQN, SAC	CQL, IQL, BCQ, Decision Transformer

记住：

- **Online $\neq$ on-policy：**Q-learning 是 online 但 off-policy
- **Offline 必然 off-policy：**数据已固定，无法再用现在的策略采样
- **On-policy + offline = 不存在**（除非定义为 supervised learning / 行为克隆）

Offline RL 的本质困难

为什么 offline 比 online 难得多？

分布偏移 (distribution shift): 你想学的 policy π_t 会去 π_t 喜欢的状态，但数据集里全是 π_b 喜欢的状态。在 π_b 从未访问过的 (s, a) 上， Q 值是凭空外推出来的——而 $\max$ 算子最喜欢挑高估值的那个外推区域，越学越离谱。

Online 的 Q-learning 不怕这个，因为它真去执行 $\arg \max$ 动作、立刻得到环境的反馈纠正。Offline 没有这个反馈渠道。

对策思路: 让学到的 policy “贴近数据分布”——这就是 CQL (保守地压低 OOD 动作的 Q 值)、IQL (避免显式 $\max$ 操作)、BCQ (限制动作只能从行为分布中选) 的共同主线。

10.4 经验回放：让 online 算法重复消化样本

大白话

TD/Q-learning 的“原生”形式：观察一个 (s, a, r, s') ，更新一次，丢掉，再观察下一个。两个浪费：

1. 样本只用一次就丢——但收集样本很贵（特别是真实机器人/医疗/工业过程）
2. 相邻样本强相关（同一轨迹里前后步状态相似），违反随机逼近定理喜欢的 i.i.d. 假设，让神经网络训练不稳

经验回放 (Experience Replay, Lin 1992; Mnih et al. 2015): 把每个观察到的 (s, a, r, s') 存进一个固定大小的 buffer，更新时从中随机抽一个 mini-batch。一颗样本被反复消化。

Algorithm 6 Q-learning with Experience Replay

- 1: 初始化 Q , replay buffer $\mathcal{B} = \emptyset$ (容量 N , 典型 $N = 10^6$)
 - 2: **for** 每步 t **do**
 - 3: 在 s_t 用 ϵ -greedy 选 a_t , 执行后观察 r_t, s_{t+1}
 - 4: $\mathcal{B} \leftarrow \mathcal{B} \cup \{(s_t, a_t, r_t, s_{t+1})\}$ (满了 FIFO 丢老的)
 - 5: 从 $\mathcal{B}$ 中均匀采样一个 mini-batch B (典型 $|B| = 32 \sim 256$)
 - 6: **for** $(s, a, r, s') \in B$ **do**
 - 7: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
 - 8: **end for**
 - 9: **end for**
-

经验回放为什么有效，以及它的前提

三个好处：

1. **样本复用**：每个 (s, a, r, s') 被使用多次（典型 $\sim 8\times$ ），样本效率近似提升一个数量级
2. **去相关**：mini-batch 里的样本来自不同时刻、不同 episode，近似 i.i.d.，更接近随机逼近定理的假设
3. **稳定梯度**：mini-batch 平均降低梯度方差，让 bootstrap 更新更稳

关键前提：算法必须是 **off-policy**。

为什么？buffer 里的数据来自**过去的策略**，按现在的 policy 可能根本不会做那些动作。on-policy 算法（SARSA, PPO）的 target 要求 a' 按当前 π 采样——但你存的是旧 π 下的 a' ，分布不对，更新方向有偏。

所以经验回放是 **Q-learning** 家族的专利，SARSA/PPO 没法直接用。这是 on/off-policy 差别最实际、最频繁出现的工程后果。

例 10.2 (Prioritized Experience Replay (PER, Schaul et al. 2016)). 均匀采样把所有样本一视同仁，但有些样本（TD 误差大的）更有学习价值——它们意味着“模型当前预测错得厉害”。

PER：按 $|\delta_i|$ 给样本分配采样概率 $p_i \propto |\delta_i|^\alpha$ ，对 $|\delta|$ 大的样本采得更频繁。需要重要性采样权重 $w_i \propto (Np_i)^{-\beta}$ 校正引入的偏差。

效果：典型场景下样本效率再提升 2 ~ 4 倍。代价：需要维护 sum-tree 数据结构，每次 update 都要重排。

10.5 目标网络：bootstrap 的稳定化器

大白话

Q-learning 的 target: $r + \gamma \max_{a'} Q(s', a')$ 。注意 target 里的 Q 和被更新的 Q 是同一个——你在追一个一直在动的靶子。

表格情形下这不要紧（每步只改一个 (s, a) ）。但用神经网络逼近 Q 时，一次梯度 step 会同时影响所有 (s, a) 处的 Q 值，target 随之大幅波动，bootstrap 的偏差被放大、训练经常发散。

目标网络 (Target Network, Mnih et al. 2015)：维护一个 Q 的滞后副本 Q_{target} ，在 target 里用 Q_{target} 而不是 Q 。

定义 10.3 (带 target network 的 Q-learning 更新)。

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q_{\text{target}}(s', a') - Q(s, a) \right]$$

Q_{target} 的两种更新方式：

- **硬更新 (hard update)**：每 C 步把 $Q_{\text{target}} \leftarrow Q$ (DQN 原版, $C = 10^4$)

- **软更新 (Polyak / soft update):** 每步 $Q_{\text{target}} \leftarrow (1 - \tau)Q_{\text{target}} + \tau Q$ ($\tau \approx 0.005$, 常用于 DDPG、SAC)

为什么这样稳定

不动的 target = 转化成了**有监督回归**。在硬更新的两次同步之间, Q_{target} 固定不动, 所以 target $r + \gamma \max Q_{\text{target}}(s')$ 对每个样本是个**常数**——拟合 Q 向它逼近就是普通的回归问题, 神经网络擅长。每过 C 步换一组新的 target, 再回归一段。

类比: 你在调一个 PID 控制器, 但 setpoint 一直在跳——调起来当然发散。把 setpoint 每段时间冻住, 反而能调进去。

10.6 Q-learning 的过估计偏差与 Double Q-learning

大白话

Q-learning 有一个隐蔽的系统性偏差: Q 值会被高估。原因是 $\max$ 算子和噪声的相互作用。

定理 10.2 (Jensen 不等式: $\max$ 的偏差). 对随机变量 $X_1, \dots, X_n$,

$$\mathbb{E}\left[\max_i X_i\right] \geq \max_i \mathbb{E}[X_i]$$

等号仅在所有 X_i 恒等时成立。**max** 是凸函数, **noise** 一过它就变正偏。

Q-learning 为什么过估计

Q-learning 的 target 含 $\max_{a'} Q(s', a')$ 。 $Q(s', a')$ 是带噪声的估计, 分解为

$$Q(s', a') = Q^*(s', a') + \varepsilon_{a'}, \quad \mathbb{E}[\varepsilon_{a'}] = 0$$

对 noise 取 $\max$:

$$\max_{a'} Q(s', a') = \max_{a'} (Q^*(s', a') + \varepsilon_{a'})$$

取期望, 由 Jensen:

$$\mathbb{E}\left[\max_{a'} Q(s', a')\right] \geq \max_{a'} Q^*(s', a')$$

即使每个 $\varepsilon_{a'}$ 都零均值, 被 $\max$ 选中的那个动作的 ε 倾向于偏正——因为 $\max$ 偏好挑大噪声的样本。

后果: target 被系统性高估 $\rightarrow Q$ 被推高 $\rightarrow$ 下次更新更高 $\rightarrow$ 一直往上漂, 策略对噪声特别敏感, 训练有时崩盘。

定义 10.4 (Double Q-learning, van Hasselt 2010). 思路: 把**选动作**和**评估动作**用两个独立的 Q 估计——一个挑出 $\arg \max$, 另一个给它估值, noise 解耦。

维护两个网络 Q_A, Q_B 。每步随机选一个更新:

$$Q_A(s, a) \leftarrow Q_A(s, a) + \alpha \left[r + \gamma Q_B(s', \arg \max_{a'} Q_A(s', a')) - Q_A(s, a) \right]$$

(再以对称形式更新 Q_B 。)

关键：用 Q_A 挑动作 ($\arg \max$)，用 Q_B 估值。 Q_B 对 Q_A 挑出来的动作给出独立的估计——max 偏差消失，因为 max 的 noise 和评估的 noise 不再相关。

Double DQN (深度版本, Hasselt et al. 2016): 用主网络挑动作、target 网络估值，几乎免费实现，是 DQN 系列的标配改进。

10.7 探索策略小结

大白话

我们已经见过几种探索方式，放在一起对比：

- **ϵ -greedy** (MC control, SARSA, Q-learning, DQN): 以 ϵ 概率随机、 $1 - \epsilon$ 概率 greedy。简单但不智能 (在已经熟悉的状态也乱来)
- **UCB 式 bonus** (UCB-VI): 把”不确定性”加到价值估计上，引导探索去不熟悉的 (s, a) 。理论好，但需要计数器
- **Boltzmann/softmax**: $\pi(a|s) \propto \exp(Q(s, a)/T)$, 温度 T 控制探索强度。比 ϵ -greedy 更平滑、对 Q 差异敏感
- **Thompson sampling** (贝叶斯 RL): 从 Q 的后验分布采样动作。Bandit 上经常打败 UCB
- **内在动机/curiosity** (深度 RL): 用状态预测误差等”惊讶度”作内在奖励——处理稀疏外部奖励的利器

通用规律：探索越智能 (用上更多结构)，样本效率越高，但实现复杂度也越高。深度 RL 实际中 ϵ -greedy + replay + target net + Double 仍是最常见的组合，简单粗暴管用。

10.8 Rollout: 把”模拟一段轨迹”独立成一个动词

大白话

”Rollout”这个词本课前面已经用过很多次但没有正式定义。它就是一个简单的动作：

Rollout = 从某个状态 s 出发、按某个策略 π 跑一段轨迹（真实或想象的），把过程中的奖励/状态/动作收集起来。

你已经在每个章节里都做过它：

- MC 评估：从 s rollout 到 episode 结束，记录 G_t
- TD(0)：从 s rollout 一步，剩下用 $V(s')$ bootstrap
- n -step return：从 s rollout n 步，剩下用 $V(s_{t+n})$ bootstrap
- UCB-VI 内部的值迭代：在估出来的模型 $\hat{P}, \hat{R}$ 上做规划——也是一种 rollout，只不过在估的模型里

Rollout 是 RL 算法的**基本动作**，独立于”学什么”和”怎么更新”。明确把它当作概念，能更清楚地看 deep RL 论文里 actor、worker、planner 这些角色。

Rollout 长度的谱系

按 rollout 长度区分 RL 算法：

长度	Target	算法
1 步	$r + \gamma V(s')$	TD(0), SARSA, Q-learning
n 步	$\sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n})$	n -step TD, Rainbow DQN
λ -加权	$\sum_n (1 - \lambda) \lambda^{n-1} G_t^{(n)}$	TD(λ), GAE
整段	G_t	MC

Bias-variance 沿 rollout 长度的谱系：

- 短 rollout：bootstrap 占比大 $\rightarrow$ 偏差大（ V 估计不准 target 就不准），方差小（少叠加奖励噪声）
- 长 rollout：真实奖励占比大 $\rightarrow$ 偏差小，方差大
- Rollout 长度本质上是一个连续可调的旋钮——TD() 通过 λ 平滑控制

Rollout 在 RL 系统里的四种用途 (1) 数据采集（最基本用法）

Online RL 标准做法：actor 进程在真实环境里 rollout 收集 (s, a, r, s') ，喂给 learner 进程更新参数。

- 同步（A2C, PPO）：一批 actor 同时 rollout 固定步数，然后 learner 一次性更新
- 异步（A3C, IMPALA, Ape-X, SEED RL）：actor 持续 rollout、learner 持续更新，互不阻

塞。需要处理”actor 用的是旧版策略”的 off-policy 校正——IMPALA 的 V-trace 算法是这一线的代表

Deep RL 实际跑起来，rollout 经常占 70%+ 时间——这是为什么 RL infra 设计的核心常常是”如何高效地并行 rollout”。

(2) Decision-time planning (MCTS)

不是把 rollout 用来训练，而是用来在当前状态下做决策。

Monte Carlo Tree Search (MCTS): 在当前状态 s 要决定动作时，模拟很多次（典型 $\sim 10^3 - 10^5$ 次）”如果我这么走、对手那么走、...”，每次跑到底（或固定深度后用 V bootstrap），统计每个动作的平均回报，选最好的。

- **AlphaGo**: 用神经网络指导 rollout 中的动作选择 (policy network)，用神经网络对叶节点估值 (value network)，UCB-style 选展开节点 (UCT)
- **AlphaZero / MuZero**: 进一步用学到的模型做 rollout (MuZero 甚至不需要环境真实 dynamics，只学一个 latent 模型让 MCTS 在里面跑)

MCTS 的精髓：每个动作的价值估计 = 多次 rollout 的样本均值——这其实是 bandit-style sample average 套了一层 tree 结构。

(3) Imaginary rollout (model-based RL)

学到一个模型 $\hat{P}, \hat{R}$ 后，可以在模型里 rollout 生成”想象的”经验，喂给标准 model-free 算法。

- **Dyna-Q** (Sutton 1990): 每次真实交互后，从模型里采样 k 步 rollout 也喂给 Q-learning。最早的 model-based + model-free 混血
- **MBPO** (Janner et al. 2019): 从 replay buffer 里的真实 s 出发，用学到的模型做短 rollout (避免长期复合误差)，把这些 imagined transitions 加进 buffer
- **Dreamer** (Hafner et al. 2020): 学一个 latent dynamics 模型，把 rollout 完全放在 latent space 做——计算便宜得多

核心权衡：模型 rollout 越长，复合误差越大（前面 §Simulation Lemma 分析过： $O(1/(1-\gamma)^2)$ 放大）；越短，越接近 model-free。所以现代 MBRL 一般用短 rollout + 真实数据混合。

(4) Rollout algorithm (Tesauro & Galperin 1996; Bertsekas)

经典的”做一步策略改进”算法：给定 base policy π_b ，要决定在状态 s 做什么。对每个候选动作 a ：

1. 假定先执行 a ，然后按 π_b rollout 到底
2. 多次重复，估计 $\hat{Q}^{\pi_b}(s, a)$ （蒙特卡洛样本均值）
3. 选 $\arg \max_a \hat{Q}^{\pi_b}(s, a)$

这等价于把策略迭代的”一步改进”做成 online 蒙特卡洛版本。**保证**：得到的策略至少和 π_b 一样好（策略改进定理）。代价：每决策一次都要做一堆 rollout，慢。

历史上 Tesauro 的 TD-Gammon 系列就是用这个把人类水平的 backgammon 程序做到超人。MCTS 可以看作它的”tree-structured”升级版。

一句话总结四种用法

- 用真实环境 rollout 收集训练数据 → online RL 主流（PPO/DQN/SAC 的 actor）
- 用 rollout 在决策时刻搜索 → MCTS（AlphaGo 系列）
- 用模型 rollout 生成合成数据 → model-based RL（Dyna/MBPO/Dreamer）
- 用 rollout 做一步策略改进 → Rollout algorithm（经典 Bertsekas 方法）

同一个动作（rollout），放在不同位置就是不同算法家族。

例 10.3 (一个状态、四种 rollout 用法的对比). 机械臂抓取任务。当前状态：手在桌面上方某处，眼前有个杯子。

(1) **训练数据 rollout** (DQN actor)：执行当前 ϵ -greedy 策略，从这个状态走到 episode 结束，把每步 (s, a, r, s') 塞进 replay buffer。

(2) **Decision-time rollout** (MCTS)：从这个状态做 10^4 次模拟（在环境模型里），统计每个动作（向左、向右、向下、抓）的平均回报，挑最好的执行。

(3) **Imaginary rollout** (MBPO)：从 buffer 里采一个真实状态，用学到的 dynamics 模型走 5 步 imaginary trajectory，把这 5 个合成的 transition 也塞进训练 batch。

(4) **Rollout algorithm** (base policy = ”总是向下”)：对每个候选动作 a ，假定先执行 a ，然后一直向下，跑 100 次估计 $\hat{Q}^{\pi_b}(s, a)$ ，挑最大的 a 执行。每步决策都重新 rollout 一遍。

同一个”rollout”动作，承担了完全不同的算法角色。

10.9 其他小技巧速览

大白话

还有几个反复出现的小技巧，不像 replay/target net 那么”必备”，但在 deep RL 论文里几乎处处可见。这一节做个速览，方便后面读论文时不发懵。

(1) **n -step return: 1-step TD 和 MC 之间的可调节插值** 回忆 TD(0) 的 target 是 $r_t + \gamma V(s_{t+1})$ （一步真实奖励 + 一步 bootstrap），MC 的 target 是 $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ （全部真实奖励）。 n -step 就在中间插值：

$$G_t^{(n)} = \underbrace{\sum_{k=0}^{n-1} \gamma^k r_{t+k}}_{n \text{ 步真实奖励}} + \underbrace{\gamma^n V(s_{t+n})}_{n \text{ 步后 bootstrap}}$$

对应的更新： $V(s_t) \leftarrow V(s_t) + \alpha [G_t^{(n)} - V(s_t)]$ 。

偏差-方差权衡： n 大 $\rightarrow$ 接近 MC，方差大偏差小； n 小 $\rightarrow$ 接近 TD(0)，偏差大方差小。实践中 $n = 3 \sim 5$ 常常比 1-step 好（Rainbow DQN 用 $n = 3$ ）。更优雅的版本是后面 §TD(λ) 的 λ -weighted 平均——所有 n 加权混合。

(2) Reward 标准化与 clipping 不同任务奖励量级差异巨大：CartPole 每步 +1，Atari 单关可上千分，机械臂任务奖励可能在 $[-100, 0]$ 。神经网络（特别是带 Adam 优化器的）对输入/输出尺度敏感，奖励量级直接影响 Q 值量级、TD 误差量级、梯度量级。

常见做法：

- **Running normalization:** 维护 r 的滑动 mean / std，用 $(r - \mu)/\sigma$ 喂给网络
- **Reward clipping:** DQN 在 Atari 上用 $\text{sign}(r) \in \{-1, 0, 1\}$ 暴力 clip——丢失幅度信息，但保留了”好/坏/无“
- **Return normalization**（PPO 等）：归一化 G_t 而非单步 r

副作用： reward clipping 可能扭曲最优策略（10 分和 100 分的区别消失了）。当不同动作的奖励差别在量级上很重要时不要 clip。

(3) Gradient clipping TD 误差偶尔会出现 outlier（比如刚开始训练时，或遇到罕见但高奖励的样本），导致梯度突然爆炸、把网络打飞。

标准对策： clip 梯度的 ℓ_2 范数：

$$\nabla \leftarrow \nabla \cdot \min\left(1, \frac{c}{\|\nabla\|_2}\right)$$

典型 $c = 0.5 \sim 10$ 。几乎所有 deep RL 实现都默认开启。

(4) Huber loss: Q-learning 的鲁棒损失 Q-learning 的标准 MSE loss 在大 TD 误差处会有过大梯度（ $\nabla = 2\delta$ ），加剧 outlier 的破坏。

Huber loss（DQN 标配）：

$$L_\kappa(\delta) = \begin{cases} \frac{1}{2}\delta^2 & |\delta| \leq \kappa \\ \kappa(|\delta| - \frac{\kappa}{2}) & |\delta| > \kappa \end{cases}$$

小误差处像 MSE（光滑、梯度精确），大误差处像 MAE（梯度有界，鲁棒）。典型 $\kappa = 1$ 。

(5) Dueling 架构： $Q = V + A$ 分头学 很多状态下”做哪个动作都差不多“——这时只学 $V(s)$ 就够，每个 $Q(s, a)$ 单独学浪费样本。

Dueling DQN (Wang et al. 2016): 网络分两个 head，分别输出 $V(s; \theta_v)$ 和 advantage $A(s, a; \theta_a)$ ，组合：

$$Q(s, a; \theta) = V(s; \theta_v) + \left(A(s, a; \theta_a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta_a) \right)$$

减均值是为了 identifiability（否则 V 和 A 间能差一个任意常数）。在动作数大、很多状态下动作选择不敏感的任务（Atari 大部分游戏）效果显著。

（这个架构和后面 §Linear FA 的 advantage 概念也呼应： $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ ，TD 误差 δ 是它的无偏估计。）

(6) 熵正则化：在 loss 里鼓励策略保持随机 动机：deterministic 策略 $\rightarrow$ 探索停止 $\rightarrow$ 容易陷入局部最优。

做法：在 loss 里加 $-\beta H(\pi(\cdot|s))$ ，其中 $H(\pi) = -\sum_a \pi(a|s) \log \pi(a|s)$ 是策略熵。 β 控制随机性强度。

- **A2C/PPO**：加一个小系数 ($\beta \sim 0.01$) 的熵 bonus 防止过早收敛
- **SAC (Soft Actor-Critic)**：把它做到极致——目标是最大化 奖励 + $\alpha \cdot$ 策略熵，称为“最大熵 RL”。这等价于把策略当作 Bellman 算子的不动点；理论上很优雅

熵正则化的副作用：最优策略不是原 MDP 的最优策略（多了一个熵项），但在 $\beta \rightarrow 0$ 极限下重合。SAC 还有 auto-tuning α 的版本。

(7) **Hindsight Experience Replay (HER)**：稀疏奖励的救星 稀疏奖励（只在最终成功时才有 +1，其他全 0）是 RL 的克星——绝大多数 trajectory 都拿 0 奖励，几乎学不到东西。

HER (Andrychowicz et al. 2017) 思路：失败的轨迹也可以当作“达成了另一个目标”的成功轨迹。

具体做法（goal-conditioned 设定下）：

- 原目标 g ，trajectory 实际终点 $s_T \neq g \rightarrow$ 失败，奖励全 0
- Relabel：把目标换成 $g' = s_T$ ，重新算奖励 $\rightarrow$ 这个轨迹“达到了” g' ，变成成功样本！
- 把 relabeled 轨迹塞进 buffer，正常 off-policy Q-learning

效果：在机械臂抓取、push、导航等 goal-conditioned 任务上，从“完全学不到”变成“几小时收敛”。

直觉：每次失败都教会你“如何到达 s_T ”，哪怕 s_T 不是你想去的地方——总有一天 s_T 可能就是某个用户的目标。把环境反馈的信息密度极大化。

(8) **Potential-based reward shaping**：加先验知识但不毁最优性 工程上经常想给奖励函数加点“提示”——比如“靠近目标更好”。但随便加 shaping 会改变最优策略，得不偿失。

Potential-based shaping (Ng, Harada & Russell 1999)：选一个势函数 $\Phi : S \rightarrow \mathbb{R}$ ，把奖励改成

$$R'(s, a, s') = R(s, a) + \gamma \Phi(s') - \Phi(s)$$

即“势的变化”。

定理 10.3 (Ng et al. 1999). Potential-based shaping 后的 MDP 与原 MDP 有相同的最优策略。

证明思路：累加 shaping 项的话，中间项全消掉，只剩 $\gamma^T \Phi(s_T) - \Phi(s_0)$ ，这是个常数（对所有策略一样），所以最优性不变。但每步的“局部信号”更密集了，学习更快。

用法：把先验知识“哪些状态更接近成功”编码进 Φ ——例如导航任务设 $\Phi(s) = -\|s - s_{\text{goal}}\|$ 。保证最优策略不变，但学习速度可能数量级提升。

(9) 其他点到为止

- **Distributional RL (C51, QR-DQN, IQN)**: 学的不是 $\mathbb{E}[G_t]$ 而是 G_t 的整个分布。直觉: 分布信息比期望多。Atari 上 SOTA 经常用
- **NoisyNet**: 把网络参数加可学习噪声, 作为 ϵ -greedy 的替代——探索量随训练自动衰减
- **Frame stacking**: Atari 输入是单帧图像 (无速度信息), 堆叠最近 4 帧 → 网络能感知运动方向。POMDP 的简单粗暴对策
- **Polyak averaging of network weights**: 保存训练过程中权重的 EMA 副本作为最终评估模型——通常比最后一步的权重稳定
- **Spectral / Layer normalization**: 稳定深度网络训练, layer norm 在 RL 中经常比 batch norm 好 (batch norm 假设 i.i.d., replay 出来的 mini-batch 不严格满足)

挑技巧的原则

不要一上来就把所有技巧都堆上。建议顺序:

1. 先把 baseline 跑通 (Q-learning + replay + target net)
2. 看 loss/Q 是否爆炸 → 加 gradient clipping、Huber
3. 看奖励量级 → 加 normalization
4. 看是否陷入局部最优 → 加熵正则化或更聪明的探索
5. 看是否过估计 → 加 Double
6. 看是否动作不敏感 → 加 Dueling
7. 看是否稀疏奖励 → HER 或 shaping
8. n -step、distributional 是锦上添花, 先调通基础再上

Deep RL 实战很大程度上就是诊断 + 加这些 trick 的循环。理论保收敛, 工程保不散。

10.10 算法地图：把这一切收进一张表

三维分类总览

算法	Model	On/off	On/offline	关键技巧
值迭代	based	—	不交互	直接用 P, R
策略迭代	based	—	不交互	评估 + 改进交替
MC 评估	free	on	online	整段 G_t 取均值
MC control	free	on	online	ϵ -greedy
UCB-VI	based	on	online	估模型 + 乐观 bonus
SARSA	free	on	online	ϵ -greedy + GLIE
Q-learning	free	off	online	max 算子
DQN	free	off	online	上面三件套 (replay + target net + Double)
SAC	free	off	online	+ 最大熵 + 双 Critic
PPO	free	on	online	信赖域 / clipping
CQL / IQL / BCQ	free	off	offline	Q 值正则 / 避免 max / 行为约束

读法：

- Model-based 通常占 offline 好处（不用一直交互），但要假设环境可模拟或模型可学
- On-policy 算法（SARSA, PPO）天生不能用 replay，只能用最近收集的数据
- Offline 强制 off-policy，且需要特殊正则避免 OOD 问题
- **Q-learning 家族**（Q-learning $\rightarrow$ DQN $\rightarrow$ SAC $\rightarrow$ CQL）是一条主线——走 off-policy 路线，逐渐加上 replay、target net、Double、最大熵、保守正则等技巧应对不同场景

后面 §Q-learning 收敛性还会从理论角度再补一刀（model-based/free 的精确分类、深一些的 online/offline 讨论），与本节互补——本节关心“工程上是什么”，下一节关心“理论上为什么收敛”。

11 Q-learning 收敛性

大白话

值迭代需要知道 P 和 R 。如果不知道呢？

Q-learning：用样本代替期望，在线更新 Q 值。

11.1 有模型 vs 无模型：核心区别

什么是“模型”？

在 RL 中，环境模型指两个东西：

1. 状态转移概率 $P(s'|s, a)$ ：在状态 s 做动作 a ，跳到 s' 的概率
2. 奖励函数 $R(s, a)$ ：在状态 s 做动作 a ，直接获得多少奖励

有模型 vs 无模型的核心区别：算法更新价值时，是否直接用到 P 和 R 的显式数学形式。

重要澄清：Model-Free 也需要“访问”环境！

很多人误以为“无模型”意味着“什么都不需要知道”。

错！Model-Free 方法（如 Q-learning）仍然需要：

- 能够在环境中执行动作
- 观察环境返回的 (r, s')
- 收集大量交互样本

它只是不需要知道 P 和 R 的数学公式，但它需要把环境当作黑盒来查询。

打个比方：

- **Model-Based**：你有一本物理教科书，可以用公式算出球会落在哪
- **Model-Free Online**：你没有教科书，但你可以反复扔球，观察它落在哪
- 两者都需要“访问”物理世界，只是方式不同

更精确的三分类

方法	需要 P, R 公式	需要在线交互	例子
Model-Based	✓	可选	值迭代、MBPO
Model-Free Online	✗	✓	Q-learning、PPO
Model-Free Offline	✗	✗	CQL、IQL、Decision Transformer

Offline RL（也叫 Batch RL）才是“真正的无模型”：

- 不需要 P 和 R 的公式
- 不需要和环境交互
- 只用预先收集好的历史数据集 $\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}$

例 11.1 (三种方法的比喻——学开车). **Model-Based**（有驾驶模拟器的物理引擎）：

- 你知道方向盘转 30 度，车会怎么转弯（有 P 的公式）

- 可以在脑子里规划：“如果我这样打方向盘，车会...”

Model-Free Online（真车上路练）：

- 你不知道转弯的物理公式
- 但你可以反复尝试，每次转完看看效果，慢慢学会
- 需要一辆真车和一条路

Model-Free Offline（看别人的行车记录仪）：

- 你不知道物理公式
- 你也没有车可以练
- 你只有一堆别人开车的视频，从中学习“什么情况该怎么做”
- **这是最难**的：你不能尝试视频里没有的动作

为什么 Offline RL 很难？

想象你只能看别人开车的视频学开车，不能自己上手：

分布偏移（Distribution Shift）：

- 视频里的司机都是老司机，从不犯错
- 你学到的策略只在“正常情况”下有效
- 一旦遇到视频里没见过的情况（比如你自己犯了个小错），你就不知道怎么纠正

过度乐观（Overestimation）：

- 数据集里没有尝试过的动作，Q 值可能被高估
- 因为你从来没见过那个动作的后果

这就是为什么 Offline RL 需要特殊算法（CQL、IQL 等）来处理这些问题。

值迭代——上帝视角

值迭代的更新公式：

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) \left[R(s, a) + \gamma V_k(s') \right]$$

为什么是有模型的？

- **显式依赖 P** ：公式里有 $\sum_{s'}$ 和 $P(s'|s, a)$ ，必须预先知道所有可能的下一状态及其概率
- **计算期望**：不是”试一试”，而是直接用概率算出期望值
- **不与环境交互**：只要给它规则书（ P 和 R ），它在脑子里就能算出最优解

比喻：就像下棋，你知道规则，可以在脑海里推演，不需要真的落子。

Q-learning——试错学习（Online Model-Free）

Q-learning 的更新公式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{目标值 (来自实际经验)}} - Q(s, a) \right]$$

为什么是无模型的？

- **P 和 R 消失了**：公式里没有 $\sum_{s'}$ ，没有 $P(s'|s, a)$
- **基于采样**：智能体实际执行动作 a ，环境反馈”你到了 s' ，奖励 r ”
- **依靠交互**：不需要知道”往右走滑倒概率 10%”，只需要摔几次就知道了

但它仍然需要：能够和环境交互，收集 (s, a, r, s') 样本！

例 11.2 (计算期望 vs 采样——全班平均身高). **值迭代（有模型）**：

老师手里有一张表（模型）：1.6m 的学生占 10%，1.7m 的占 20%...

直接用公式计算： $1.6 \times 0.1 + 1.7 \times 0.2 + \dots = \text{期望身高}$

不需要看具体学生，只要有概率表就行。

Q-learning（无模型）：

老师手里没表。随机抓一个学生量身高（采样），1.65m，更新估计值；

再抓一个，1.75m，再更新...

不需要概率表，需要一个个具体的样本。

抓的学生足够多，估计值就逼近真实平均身高。

特性	值迭代	Q-learning
类型	Model-Based（动态规划）	Model-Free（时序差分）
核心机制	计算期望（Expectation）	采样（Sampling）
是否需要 $P(s' s, a)$	是（必须已知）	否（它是未知黑盒）
数据来源	环境的数学定义	实际交互的经验样本
比喻	看着地图规划路线	在迷宫里瞎逛摸索路线

11.2 Q-learning 算法

例 11.3 (Q-learning 的生活比喻——试吃学习). 你搬到新城市, 想找好吃的餐厅。

问题: 你不知道每家店多好吃（不知道 R ），也不知道吃完会想去哪（不知道 P ）。

Q-learning 策略:

1. 随便选一家店吃（探索）
2. 吃完打个分（观察奖励 r ）
3. 更新你对这家店的评分:

$$Q(\text{这家店}) \leftarrow Q(\text{这家店}) + \alpha \times (\text{实际体验} - \text{之前预期})$$

关键: 你不需要知道”这家店平均多好吃”，只需要每次吃完更新一点点！
吃得多了，你的评分 Q 自然会收敛到真实水平。

Algorithm 7 Q-learning

- 1: 初始化 $Q(s, a)$ 任意
 - 2: **for** 每步 t **do**
 - 3: 在状态 s_t 选择动作 a_t (ϵ -greedy)
 - 4: 观察 r_t 和 s_{t+1}
 - 5: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t [r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$
 - 6: **end for**
-

例 11.4 (Q-learning 更新的直觉——修正预期). 你以为某餐厅 8 分 ($Q = 8$)，结果吃完发现:

- 这顿饭 7 分 ($r = 7$)
- 吃完心情好，下一个状态价值 10 分 ($\max_{a'} Q(s', a') = 10$)
- $\gamma = 0.9$

TD 目标: $r + \gamma \max Q' = 7 + 0.9 \times 10 = 16$

TD 误差: $16 - 8 = 8$ (实际比预期好!)

更新 ($\alpha = 0.1$): $Q \leftarrow 8 + 0.1 \times 8 = 8.8$

你把评分从 8 上调到 8.8。多吃几次，评分会稳定在真实水平。

定理 11.1 (Q-learning 收敛 - Watkins & Dayan, 1992; Jaakkola et al., 1994). 如果:

1. 所有 (s, a) 被无限次访问
2. 学习率满足: $\sum_t \alpha_t(s, a) = \infty$ 且 $\sum_t \alpha_t(s, a)^2 < \infty$

则 $Q_t \rightarrow Q^*$ 以概率 1。

例 11.5 (学习率条件的直觉). 条件 1: $\sum_t \alpha_t = \infty$ (学习率之和发散)

意思: 你要持续学习, 不能太早停下来。

如果 α_t 衰减太快 (比如 $\alpha_t = 1/t^2$), 总学习量有限, 可能还没学到真值就停了。

条件 2: $\sum_t \alpha_t^2 < \infty$ (学习率平方之和收敛)

意思: 你要逐渐冷静, 不能总是大幅调整。

如果 α_t 不衰减 (比如恒等于 1), 噪声会一直影响你, 永远不稳定。

黄金法则: $\alpha_t = 1/t$ 刚好满足两个条件!

证明. Step 1: 写成随机逼近形式

Q-learning 更新可以写成:

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha_t(s, a) \cdot \mathbf{1}_{(s_t, a_t)=(s, a)} \cdot [r_t + \gamma \max_{a'} Q_t(s_{t+1}, a') - Q_t(s, a)]$$

定义:

- $F_t(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q_t(s', a') - Q_t(s, a)$ (期望更新方向)
- $w_t(s, a) = [r_t + \gamma \max_{a'} Q_t(s_{t+1}, a')] - [R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q_t(s', a')]$ (噪声)

则更新变成:

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha_t \cdot \mathbf{1}_{(s_t, a_t)=(s, a)} \cdot [F_t(s, a) + w_t(s, a)]$$

Step 2: 噪声是鞅差

条件于历史 $\mathcal{F}_t = \sigma(s_0, a_0, r_0, \dots, s_t, a_t)$:

$$\mathbb{E}[w_t(s, a) | \mathcal{F}_t] = 0$$

因为 r_t 和 s_{t+1} 的期望正好是 $R(s, a)$ 和 $\sum_{s'} P(s'|s, a)(\cdot)$ 。

Step 3: F_t 具有收缩性质

令 $\Delta_t = Q_t - Q^*$ 。由于 $Q^* = T^*Q^*$ (Bellman 最优方程):

$$\begin{aligned} F_t(s, a) &= (T^*Q_t)(s, a) - Q_t(s, a) \\ &= [(T^*Q_t)(s, a) - (T^*Q^*)(s, a)] + [Q^*(s, a) - Q_t(s, a)] \\ &= [(T^*Q_t)(s, a) - (T^*Q^*)(s, a)] - \Delta_t(s, a) \end{aligned}$$

由 T^* 的收缩性 (前面证明过):

$$|(T^*Q_t)(s, a) - (T^*Q^*)(s, a)| \leq \gamma \|\Delta_t\|_\infty$$

所以：

$$|F_t(s, a) + \Delta_t(s, a)| \leq \gamma \|\Delta_t\|_\infty$$

Step 4: 应用随机逼近定理

我们需要 Jaakkola 等人的随机逼近定理：

定理 11.2 (随机逼近收敛条件). 考虑迭代: $x_{t+1} = x_t + \alpha_t(F_t(x_t) + w_t)$

如果：

1. $\mathbb{E}[w_t | \mathcal{F}_t] = 0$, $\mathbb{E}[w_t^2 | \mathcal{F}_t] \leq C(1 + \|x_t\|^2)$
2. 存在 $\gamma < 1$ 使得 $\|F_t(x)\| \leq \gamma \|x\|$ (收缩性)
3. $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$

则 $x_t \rightarrow 0$ w.p.1。

对 Q-learning, 令 $x_t = \Delta_t = Q_t - Q^*$:

- 条件 1: 噪声 w_t 有界方差 (因为 Q_t 有界, r_t 有界) ✓
- 条件 2: $F_t(\Delta) = (T^*Q) - Q = (T^*(Q^* + \Delta)) - (Q^* + \Delta)$ 满足 $\|F_t + \Delta\| \leq \gamma \|\Delta\|$ ✓
- 条件 3: 学习率条件 ✓

因此 $\Delta_t \rightarrow 0$, 即 $Q_t \rightarrow Q^*$ 。

Step 5: 处理异步更新

上面假设每步更新所有 (s, a) 。实际 Q-learning 只更新访问到的 (s, a) 。

关键: 如果每个 (s, a) 被无限次访问, 它的有效学习率序列 $\{\alpha_{t_i}\}$ 仍满足条件 3。

由独立的随机逼近定理, 每个 (s, a) 的 Q 值都收敛到 Q^* 。 □

Q-learning 收敛速度

理论: $\mathbb{E}[\|\Delta_t\|^2] = O(1/t)$ (使用 $\alpha_t = 1/t$)

实际例子: $|S| = 100$, $|A| = 4$, $\gamma = 0.9$

要达到 $\|\Delta\| \leq 0.1$:

- 需要每个 (s, a) 被更新约 $O(1/0.1^2) = 100$ 次
- 总共 $100 \times 4 \times 100 = 40000$ 次更新
- 如果探索均匀, 需要约 40000 次转移

实际中由于探索不均匀, 可能需要更多。

Q-learning vs 值迭代

	值迭代	Q-learning
需要模型	是	否
更新方式	同步（所有状态）	异步（访问到的）
收敛速度	$O(\gamma^k)$ （线性）	$O(1/k)$ （次线性）
每次代价	$O(S ^2 A)$	$O(1)$

Q-learning 牺牲了收敛速度，换来了不需要模型的优势。

12 线性函数逼近的收敛性

大白话

当 $|S|$ 很大（比如 10^6 ），不能存储完整的 $V(s)$ 表。

解决方案：函数逼近 $V_\theta(s) = \theta^T \phi(s)$

问题：还能收敛吗？

12.1 Linear TD(0)

定义 12.1 (Linear TD(0)).

$$\theta_{t+1} = \theta_t + \alpha_t \underbrace{(r_t + \gamma \theta_t^T \phi(s_{t+1}) - \theta_t^T \phi(s_t))}_{\text{TD 误差 } \delta_t} \phi(s_t)$$

定理 12.1 (Linear TD(0) 收敛 - Tsitsiklis & Van Roy, 1997). 在 on-policy 采样下，Linear TD(0) 收敛到投影 Bellman 方程的解：

$$\Phi \theta^* = \Pi_D T^\pi (\Phi \theta^*)$$

其中 Π_D 是在状态分布 D 加权下到特征空间的投影。

投影 Bellman 方程

真实 V^π 可能不在线性空间里。TD(0) 找的是：在能表示的函数中，最接近满足 Bellman 方程的那个。

证明. Step 1: 期望更新

θ 的期望更新方向：

$$\mathbb{E}[\delta_t \phi(s_t)] = \Phi^T D (T^\pi \Phi \theta - \Phi \theta)$$

Step 2: 不动点

令期望更新为零： $\Phi^T D (T^\pi \Phi \theta^* - \Phi \theta^*) = 0$

这意味着 $T^\pi \Phi \theta^* - \Phi \theta^* \perp \text{col}(\Phi)$ 在 D -内积下，即 $\Phi \theta^* = \Pi_D T^\pi \Phi \theta^*$ 。

Step 3: $\Pi_D T^\pi$ 是收缩

$$\|\Pi_D T^\pi V - \Pi_D T^\pi V'\|_D \leq \|T^\pi V - T^\pi V'\|_D \leq \gamma \|V - V'\|_D$$

由 Banach 定理, θ^* 存在且唯一。

□

定理 12.2 (逼近误差界).

$$\|V_{\theta^*} - V^\pi\|_D \leq \frac{1}{\sqrt{1-\gamma^2}} \min_{\theta} \|V^\pi - \Phi\theta\|_D$$

误差放大

γ	放大因子 $\frac{1}{\sqrt{1-\gamma^2}}$	含义
0.9	2.3	最佳逼近误差放大 2.3 倍
0.99	7.1	放大 7 倍
0.999	22.4	放大 22 倍

γ 越大, 线性逼近误差放大越严重!

12.2 Deadly Triad: 发散的危險

致命三角

以下三者组合可能导致发散:

1. 函数逼近 (不是表格)
2. Bootstrapping (用估计更新估计)
3. Off-policy (行为策略 $\neq$ 目标策略)

例 12.1 (Deadly Triad 的生活比喻——谣言传播). 想象一个办公室里传谣言:

函数逼近: 你不认识所有人, 只能根据“职位相似的人想法差不多”来猜测。

Bootstrapping: 你从别人那听到消息, 就信以为真, 然后传给下一个人。

Off-policy: 你听的是 A 部门的消息, 但你在 B 部门传。

单独一个问题不大:

- 只有函数逼近: 你直接问每个人, 虽然有猜测但能纠正
- 只有 bootstrapping: 消息来源准确, 传着传着会稳定
- 只有 off-policy: 至少你传的是真实消息

三个一起就炸了:

- 你猜 B 部门的人和 A 部门某人想法一样
- 你从 A 部门听到一个猜测 (不是事实)
- 你把这个猜测传给 B 部门, B 部门的人又传回 A 部门
- 误差越传越大, 最后谣言离谱到天上去!

例 12.2 (Baird 反例). Baird (1995) 构造了一个 7 状态 MDP, 使用 Linear Q-learning (off-policy), 参数 $\theta \rightarrow \infty$!

原因: off-policy 采样分布与更新目标不匹配, 更新方向不是下降方向。

定理 12.3 (Gradient TD 收敛 - Sutton et al., 2009). GTD2 算法通过显式最小化投影 Bellman 误差, 在 off-policy 下也收敛。

如何避免 Deadly Triad

1. 用表格: 如果状态空间小, 直接用表格, 不用函数逼近
2. 用 Monte Carlo: 不用 bootstrapping, 等 episode 结束再更新
3. 用 on-policy: 只学习你实际执行的策略 (如 SARSA)
4. 用 GTD: 如果必须三者都要, 用 Gradient TD 类方法

13 两个关键引理

大白话

接下来介绍两个在 RL 理论中反复出现的引理, 它们是后续很多结果的基础。

13.1 Simulation Lemma: 模型误差如何传播

大白话

Model-based RL 学习环境模型 $\hat{P}$, 然后在模型上规划。

问题: 模型不完美, 误差如何影响最终策略?

例 13.1 (Simulation Lemma 的生活版——用地图导航). 你要从 A 到 B, 用的是一张旧地图 (模型 $\hat{P}$)。

问题: 地图和实际道路 (真实 P) 有些出入, 会导致多大问题?

直觉分析:

- 如果只走 1 步, 地图误差 ϵ 就是实际误差 ϵ
- 如果走 2 步, 第 1 步的误差让你到错误位置, 第 2 步在错误位置又犯错
- 走 n 步, 误差累积!

更糟糕的是: 未来的误差要折现回今天。

如果 $\gamma = 0.99$ (很看重未来), 折现因子是 $1/(1 - 0.99)^2 = 10000$!

结论: 地图误差 1%, 导航效果可能差 100 倍!

定理 13.1 (Simulation Lemma). 设真实 MDP 为 (P, R) , 学习到的模型为 $(\hat{P}, \hat{R})$, $\hat{\pi}$ 是模型上的最优策略。

则在真实环境中：

$$V^*(s) - V^{\hat{\pi}}(s) \leq \frac{2}{(1-\gamma)^2} \left(\epsilon_R + \gamma \epsilon_P \cdot \frac{R_{max}}{1-\gamma} \right)$$

其中 $\epsilon_R = \max_{s,a} |R - \hat{R}|$, $\epsilon_P = \max_{s,a} \|P - \hat{P}\|_1$ 。

例 13.2 (误差放大的数值感受). $\gamma = 0.99$, $R_{max} = 1$, 模型误差 $\epsilon_P = \epsilon_R = 0.01$ (1%)

放大因子: $\frac{2}{(1-0.99)^2} = \frac{2}{0.0001} = 20000$

策略次优性上界: $20000 \times (0.01 + 0.99 \times 0.01 \times 100) \approx 20000$

1% 的模型误差，策略可能差 20000 分！

这就是为什么 model-based RL 对模型精度要求极高。

证明. **Step 1:** 分解误差

$$V^* - V^{\hat{\pi}} = \underbrace{(V^* - \hat{V}^*)}_{\text{模型乐观/悲观}} + \underbrace{(\hat{V}^{\hat{\pi}} - V^{\hat{\pi}})}_{\text{策略在模型 vs 真实}}$$

Step 2: 界定单策略误差

对任意策略 π ，比较 V^π (真实) 和 $\hat{V}^\pi$ (模型)：

$$|V^\pi(s) - \hat{V}^\pi(s)| \leq \frac{\epsilon_R}{1-\gamma} + \frac{\gamma \epsilon_P R_{max}}{(1-\gamma)^2}$$

Step 3: 合并两项。 □

13.2 为什么 Model-Based RL 在实践中常常不如 Model-Free?

一个尖锐的观察

理论上，Model-Based RL 应该更好：样本效率高、能规划、懂因果。

但实践中，Model-Free 方法 (PPO, SAC, DQN) 占据绝对主导：ChatGPT 的 RLHF、OpenAI Five 打 Dota、机器人控制...

为什么？因为 MBRL 有四大痛点，让它”一顿操作猛如虎，一看效果二百五”。

痛点 1：复合误差——最致命的杀手

这就是 Simulation Lemma 警告我们的！

场景：你的模型精度 99% (已经很高了)，需要规划 100 步。

步数	累积精度
1	0.99
10	$0.99^{10} = 0.90$
50	$0.99^{50} = 0.61$
100	$0.99^{100} = 0.36$

结果：智能体在脑海里规划出完美路径，现实中走到第 10 步就掉沟里了。

Model-Free 没这问题：它不预测未来，只记住”这个状态做这个动作大概率是好的”。不需要多步推演，没有误差累积。

痛点 2: Vapnik 原则——不要解决更难的问题

统计学习大佬 Vapnik 说: ” 尽量直接解决问题, 而不要通过解决一个更难的问题来解决它。”

MBRL 的做法 (更难的问题):

- 试图学会整个世界的物理规律 $P(s'|s, a)$
- 为了学会打网球, 先学空气动力学、重力、肌肉力学, 然后在脑子里模拟
- 这太难了!

MFRL 的做法 (直接的问题):

- 试图学会怎么赢 (策略 $\pi(a|s)$)
- 不管空气动力学, 只知道” 球飞过来, 挥拍就能赢”

现实: 策略往往很简单 (” 见到悬崖就左转”), 但环境动力学极其复杂 (悬崖边的风速、地面摩擦力)。非要建模那个复杂环境, 属于舍近求远。

痛点 3: 目标不匹配——预测准 $\neq$ 拿分高

训练环境模型用的是 MSE 损失: 让预测画面和真实画面像素越接近越好。

例子: 玩《超级马里奥》

模型的重点: 屏幕背景有大量云朵和树叶, 模型花 90% 力气预测云朵怎么飘 (像素多, MSE 权重高)

被忽略的细节: 屏幕角落有一个只有几像素的子弹飞过来。模型觉得预测这个子弹误差才几个像素, 无所谓。

后果: 模型能完美生成背景风景画, 但完全忽略了致命子弹。智能体觉得前面安全, 一走过去就挂了。

这就是: **Model 预测得准 (Loss 低)**, 但对 **RL 任务 (Reward)** 没帮助。

痛点 4: 幻觉作弊——智能体很鸡贼

智能体会利用模型的 **Bug** 刷分, 而不是学会真正的技能。

案例: 模拟器里墙壁碰撞检测有点 Bug。

智能体发现: 以某个奇怪角度撞墙, 能瞬移或获得无限分!

结果:

- 在模型里: 表现得像神, 拿到正无穷奖励
- 部署到真实环境: 只会傻傻往墙上撞, 极其弱智

这就是为什么 Sim-to-Real (仿真到现实) 如此困难。

例 13.3 (MBRL 的复活之路——隐空间模型). 既然预测像素 (真实物理世界) 太难, 那就不预

测像素！

MuZero / AlphaZero:

- 不预测棋盘画面，而是预测”价值”和”策略”
- 学会了一个”抽象的规则”
- 在棋类和 Atari 游戏中封神

Dreamer (V1/V2/V3):

- 不预测具体像素，而是在压缩的 Latent Space（潜空间）里”做梦”
- 大大减少了复合误差

结论：

- ”显式”建模物理世界（Old School MBRL）→ 不太行，世界太复杂
- ”隐式”建模（Latent MBRL）→ 正在成为 SOTA，但需要极高算法技巧
- 目前 PPO 这种”傻瓜式”Model-Free 算法仍然最好用、最稳定

对比	Model-Based	Model-Free
样本效率	高（理论上）	低
复合误差	致命问题	无
实现难度	高	低
稳定性	差	好
工业应用	少	主流（PPO, SAC）

13.3 Performance Difference Lemma

定理 13.2 (Performance Difference Lemma - Kakade & Langford, 2002).

$$V^{\pi'}(s) - V^{\pi}(s) = \frac{1}{1-\gamma} \mathbb{E}_{s' \sim d_s^{\pi'}} [\mathbb{E}_{a \sim \pi'} [A^{\pi}(s', a)]]$$

其中 $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ 是优势函数。

例 13.4 (Performance Difference 的生活版——换工作). 你当前的工作策略是 π （比如”遇到问题就加班”）。

你考虑新策略 π' （比如”遇到问题先求助同事”）。

问：新策略比旧策略好多少？

优势函数 $A^{\pi}(s, a)$ 的意思：

- 在状态 s （比如”遇到 bug”）
- 如果用旧策略，期望收益是 $V^{\pi}(s)$ （加班解决，累但有效）
- 如果这次选动作 a （求助同事），期望收益是 $Q^{\pi}(s, a)$

- $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ 就是”这次选 a 比按旧习惯好多少”

Performance Difference Lemma 说:

新策略的总收益 - 旧策略的总收益 = 新策略访问的所有状态上, 新策略的动作比旧策略平均好多少。

关键洞察: 用旧策略的 A^π 评估新策略 π' !

证明. **Step 1:** 展开 $V^{\pi'}$ 并加減 V^π

$$\begin{aligned} V^{\pi'}(s) &= \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \\ &= \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t (r_t + V^\pi(s_t) - V^\pi(s_t)) \right] \\ &= \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)) \right] + V^\pi(s) \end{aligned}$$

最后用了 telescoping: $\sum_t \gamma^t (V^\pi(s_t) - \gamma V^\pi(s_{t+1})) = V^\pi(s_0)$ 。

Step 2: 识别优势函数

$$r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t) = Q^\pi(s_t, a_t) - V^\pi(s_t) = A^\pi(s_t, a_t)$$

所以 $V^{\pi'}(s) - V^\pi(s) = \mathbb{E}_{\pi'} [\sum_t \gamma^t A^\pi(s_t, a_t)]$ 。

Step 3: 重写为状态分布的期望。 □

推论 13.1 (策略改进条件). 如果对所有 s : $\mathbb{E}_{a \sim \pi'} [A^\pi(s, a)] \geq 0$, 则 $V^{\pi'} \geq V^\pi$ 。

例 13.5 (策略改进的直觉). 如果新策略 π' 在**每个状态**的动作都不比旧策略差 ($A^\pi \geq 0$), 那新策略整体更好。

这就像: 如果你每个决策都至少和以前一样好, 那你的人生不会变差!

Performance Difference Lemma 的应用

- 策略迭代: 贪婪策略保证 $A^\pi(s, \pi'(s)) \geq 0$
- TRPO/PPO (下两周): 用 A^π 构造代理目标, 约束 π' 不离 π 太远

14 Multi-Agent RL 的收敛性

大白话

当有多个 agent 同时学习时:

- 每个 agent 的最优策略依赖于其他 agent
- 其他 agent 也在学习, 环境非平稳
- 目标是纳什均衡, 不是单方最优

例 14.1 (Multi-Agent 的生活版——室友博弈). 你和室友共用一个厨房，每天晚上都要决定：

动作：{早点做饭 (6pm), 晚点做饭 (8pm)}

收益：

- 两人都 6pm：厨房拥挤，各得 3 分
- 两人都 8pm：厨房拥挤，各得 3 分
- 一人 6pm 一人 8pm：都舒服，各得 5 分

问题：你应该选什么时间？

困难：你的最优选择取决于室友选什么！

- 如果室友选 6pm，你应该选 8pm
- 如果室友选 8pm，你应该选 6pm
- 但室友也在想同样的问题...

纳什均衡：双方都随机选（各 50%），期望收益 4 分。谁都不想单方面改变。

定义 14.1 (随机博弈). n 人随机博弈 $(S, \{A_i\}, P, \{R_i\}, \gamma)$ ：每个玩家 i 有自己的动作空间 A_i 和奖励函数 R_i 。

定义 14.2 (纳什均衡). $(\pi_1^*, \dots, \pi_n^*)$ 是纳什均衡，如果没有玩家能通过单方面改变策略获益。

例 14.2 (纳什均衡的直觉——交通灯). 两辆车在十字路口相遇，都想先过。

策略：{冲过去, 让一下}

收益矩阵（你的收益）：

	对方冲	对方让
你冲	-100（撞车）	10（你先过）
你让	5（对方先过）	0（都让，尴尬）

纳什均衡：一个冲一个让。如果你知道对方会让，你就冲；如果你知道对方会冲，你就让。在这个均衡点，谁都不想单方面改变！

14.1 零和博弈：Minimax 收敛

定义 14.3 (零和随机博弈). 两人零和随机博弈： $R_1(s, a_1, a_2) = -R_2(s, a_1, a_2)$ 。

玩家 1 最大化，玩家 2 最小化。

定理 14.1 (Minimax 定理 - von Neumann). 对任意矩阵博弈 M ：

$$\max_{\pi_1 \in \Delta(A_1)} \min_{a_2 \in A_2} \sum_{a_1} \pi_1(a_1) M(a_1, a_2) = \min_{\pi_2 \in \Delta(A_2)} \max_{a_1 \in A_1} \sum_{a_2} \pi_2(a_2) M(a_1, a_2)$$

这个共同值记为 $\text{val}(M)$ 。

定理 14.2 (Minimax-Q 收敛 - Littman, 1994). Minimax-Q 学习更新:

$$Q(s, a_1, a_2) \leftarrow (1 - \alpha)Q(s, a_1, a_2) + \alpha[r + \gamma \text{val}(Q(s', \cdot, \cdot))]$$

在满足标准随机逼近条件下, $Q \rightarrow Q^*$ (纳什均衡 Q 函数)。

证明. **Step 1: 定义 Minimax Bellman 算子**

$$(TQ)(s, a_1, a_2) = R(s, a_1, a_2) + \gamma \sum_{s'} P(s'|s, a_1, a_2) \text{val}(Q(s', \cdot, \cdot))$$

Step 2: 证明 T 是 γ -收缩

关键引理: $\text{val}(\cdot)$ 是非扩张的 (non-expansive)。

引理 14.1. 对任意矩阵 M, M' : $|\text{val}(M) - \text{val}(M')| \leq \|M - M'\|_\infty$

引理证明. 设 π_1^* 是 M 的最优混合策略, π_2^* 是 M' 的最优混合策略。

$$\begin{aligned} \text{val}(M) &= \max_{\pi_1} \min_{a_2} \pi_1^T M e_{a_2} = \min_{a_2} (\pi_1^*)^T M e_{a_2} \\ &\leq \min_{a_2} (\pi_1^*)^T M' e_{a_2} + \|M - M'\|_\infty \\ &\leq \max_{\pi_1} \min_{a_2} \pi_1^T M' e_{a_2} + \|M - M'\|_\infty \\ &= \text{val}(M') + \|M - M'\|_\infty \end{aligned}$$

对称地, $\text{val}(M') \leq \text{val}(M) + \|M - M'\|_\infty$ 。

所以 $|\text{val}(M) - \text{val}(M')| \leq \|M - M'\|_\infty$ 。 □

现在证明 T 是收缩:

对任意 Q, Q' 和 (s, a_1, a_2) :

$$\begin{aligned} |(TQ)(s, a_1, a_2) - (TQ')(s, a_1, a_2)| &= \gamma \left| \sum_{s'} P(s'|s, a_1, a_2) [\text{val}(Q(s', \cdot, \cdot)) - \text{val}(Q'(s', \cdot, \cdot))] \right| \\ &\leq \gamma \sum_{s'} P(s'|s, a_1, a_2) |\text{val}(Q(s', \cdot, \cdot)) - \text{val}(Q'(s', \cdot, \cdot))| \\ &\leq \gamma \sum_{s'} P(s'|s, a_1, a_2) \|Q(s', \cdot, \cdot) - Q'(s', \cdot, \cdot)\|_\infty \\ &\leq \gamma \|Q - Q'\|_\infty \end{aligned}$$

Step 3: Banach 不动点定理

T 是 γ -收缩, 所以存在唯一不动点 Q^* , 且值迭代收敛。

Step 4: 随机逼近收敛

Minimax-Q 是 T 的随机逼近版本。设 $\Delta_t = Q_t - Q^*$:

$$\Delta_{t+1}(s, a_1, a_2) = (1 - \alpha_t) \Delta_t(s, a_1, a_2) + \alpha_t [\gamma (\text{val}(Q_t) - \text{val}(Q^*)) + w_t]$$

由 val 的非扩张性: $|\text{val}(Q_t) - \text{val}(Q^*)| \leq \|\Delta_t\|_\infty$

这是带噪声的收缩迭代, 由随机逼近理论收敛。 □

石头剪刀布的值

收益矩阵（玩家 1）：

$$M = \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}$$

求 $\text{val}(M)$ ：设玩家 1 策略 $\pi = (p, q, 1 - p - q)$ 。

对手最优响应：选使 $\pi^T M e_j$ 最小的 j 。

$$\pi^T M e_1 = q - (1 - p - q) = p + 2q - 1$$

$$\pi^T M e_2 = -p + (1 - p - q) = 1 - 2p - q$$

$$\pi^T M e_3 = p - q$$

要最大化 $\min_j \pi^T M e_j$ ，令三者相等：

$$p + 2q - 1 = 1 - 2p - q = p - q$$

从第一和第三个： $p + 2q - 1 = p - q$ ，得 $3q = 1$ ， $q = 1/3$ 。

从第二和第三个： $1 - 2p - q = p - q$ ，得 $1 = 3p$ ， $p = 1/3$ 。

所以 $\pi^* = (1/3, 1/3, 1/3)$ ， $\text{val}(M) = 1/3 - 1/3 = 0$ 。

纳什均衡：双方都用 $(1/3, 1/3, 1/3)$ ，值为 0（公平博弈）。

14.2 一般和博弈：收敛困难

定理 14.3 (独立 Q-learning 可能不收敛)。存在一般和博弈，独立 Q-learning 在策略空间中循环。

例 14.3 (Shapley 博弈)。

	A	B	C
A	(0, 0)	(1, 0)	(0, 1)
B	(0, 1)	(0, 0)	(1, 0)
C	(1, 0)	(0, 1)	(0, 0)

独立 Q-learning 在 $(A, A) \rightarrow (B, B) \rightarrow (C, C) \rightarrow (A, A)$ 循环，永远不收敛到混合均衡 $(1/3, 1/3, 1/3)$ 。

15 TD(λ) 与资格迹

大白话

TD(0) 只看一步： $V(s) \leftarrow V(s) + \alpha(r + \gamma V(s') - V(s))$

Monte Carlo 看整条轨迹： $V(s) \leftarrow V(s) + \alpha(G_t - V(s))$

TD(λ) 用 $\lambda \in [0, 1]$ 在两者之间平滑过渡。

例 15.1 (TD(0) vs MC vs TD(λ)——考试成绩预测)。你想预测学生的期末成绩，观察到一个学生的学习轨迹：

第 1 周作业：80 分 $\rightarrow$ 第 2 周作业：85 分 $\rightarrow$ 第 3 周测验：90 分 $\rightarrow$ 期末：88 分

TD(0) (只看下一步):

- 预测第 1 周价值时, 只看第 2 周: $V(\text{第 1 周}) \approx 80 + \gamma \times V(\text{第 2 周})$
- 问题: 如果第 2 周的预测不准, 误差会传播

Monte Carlo (看整条轨迹):

- 等到期末出分, 用真实累积: $V(\text{第 1 周}) = 80 + 0.9 \times 85 + 0.81 \times 90 + \dots$
- 问题: 必须等到 episode 结束, 而且方差大

TD(λ) (综合考虑):

- 给不同远近的信号加权: 近的权重高 (更可靠), 远的权重低
- $\lambda = 0$: 退化为 TD(0)
- $\lambda = 1$: 退化为 MC
- $\lambda = 0.9$: 通常效果最好

定义 15.1 (λ -回报).

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

是所有 n -步回报 $G_t^{(n)} = \sum_{i=0}^{n-1} \gamma^i r_{t+i} + \gamma^n V(s_{t+n})$ 的指数加权平均。

例 15.2 (λ -回报的直觉——综合多个消息源). 你想预测明天的天气, 有几个信息源:

- 今晚的天气预报 (1 步预测): 80% 准确
- 看云彩自己判断 (2 步预测): 70% 准确
- 农历节气 (长期预测): 60% 准确

TD(0): 只信天气预报

MC: 只信农历节气 (等到结果出来)

TD(λ): 综合所有信息, 近期信息权重高:

$$\text{预测} = 0.3 \times \text{天气预报} + 0.21 \times \text{云彩} + 0.15 \times \text{节气} + \dots$$

($\lambda = 0.7$ 时的权重分配)

定理 15.1 (前向 = 后向). **前向视角**: 更新目标是 G_t^λ

后向视角: 用资格迹 $e(s) \leftarrow \gamma \lambda e(s) + \mathbf{1}_{s_t=s}$

两者等价! 关键是 $G_t^\lambda - V(s_t) = \sum_{k=0}^{\infty} (\gamma \lambda)^k \delta_{t+k}$

例 15.3 (资格迹的直觉——功劳分配). 你的足球队赢了比赛, 怎么分配功劳?

TD(0): 只奖励进球的人

MC: 平均分配给所有人

TD(λ) (资格迹):

- 刚传球的人，功劳大（资格迹高）
- 10 分钟前传球的人，功劳小一点（资格迹衰减）
- 上半场的助攻，功劳很小（资格迹已经很低）

资格迹就是记忆：记住谁最近访问过，用 $\gamma\lambda$ 的速度遗忘。

λ 的选择

λ	偏差	方差	适用场景
0 (TD(0))	高	低	模型准确，想快速学习
1 (MC)	低	高	Episode 短，奖励稀疏
0.9	中等	中等	通常最佳选择

16 LSTD/LSPI：批量方法

大白话

TD 是增量更新。能不能收集一堆数据，一次性算出最优参数？

LSTD：闭式解，无需调学习率！

例 16.1 (LSTD vs TD 的比喻——学习骑自行车). **TD 方法**（在线学习）：

- 骑一小段，摔了，调整一点姿势
- 再骑一小段，又摔了，再调整一点
- 每次只根据当前这一跤调整
- 问题：调整幅度（学习率）很难选，太大会乱晃，太小学得慢

LSTD 方法（批量学习）：

- 先随便骑，录下视频，摔 100 跤
- 回家看完所有视频，一次性总结出最佳姿势
- 直接用公式算，不需要调学习率
- 优点：数据利用效率高；缺点：要等收集完数据

定理 16.1 (LSTD 解). $\theta^* = A^{-1}b$ ，其中：

$$A = \mathbb{E}[\phi(s)(\phi(s) - \gamma\phi(s'))^T], \quad b = \mathbb{E}[\phi(s)r]$$

例 16.2 (LSTD 公式的直觉——最小二乘). LSTD 本质上是在解一个最小二乘问题：

” 找一个线性函数 $V_\theta(s) = \theta^T \phi(s)$ ，使得它尽可能满足 Bellman 方程”

就像高中学的线性回归：给定数据点，找一条直线最好地拟合它们。

只不过这里的”数据点”是 $(s, r + \gamma V(s'))$ ，我们要拟合的是 Bellman 方程。

LSTD 的优势

1. 数据高效：固定数据集，不需要在线交互
2. 无学习率：直接求解，不用调参
3. Off-policy：数据可来自任何策略

代价： $O(d^2)$ 内存， $O(d^3)$ 计算（ d 是特征维度）。

例 16.3 (什么时候用 LSTD). 适合 LSTD 的场景：

- 你有一批历史数据（比如过去的用户行为日志）
- 不能在线交互（比如医疗决策，不能随便试）
- 特征维度不太高（ $d < 1000$ ）

不适合 LSTD 的场景：

- 可以持续收集新数据
- 特征维度很高（ $d > 10000$ ，矩阵求逆太慢）
- 需要实时更新

17 平均奖励 MDP

大白话

之前是折扣奖励 $\sum_t \gamma^t r_t$ 。另一种设定：平均奖励 $\lim_{T \rightarrow \infty} \frac{1}{T} \sum_t r_t$ 。
对“无限期运行”的系统更自然：服务器调度、库存管理等。

定义 17.1 (平均奖励).

$$\rho^\pi = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} r_t \right]$$

定理 17.1 (平均奖励 Bellman 方程).

$$\rho^* + h^*(s) = \max_a \left[R(s, a) + \sum_{s'} P(s'|s, a) h^*(s') \right]$$

其中 h^* 是差分函数。

定理 17.2 (R-learning 收敛). R-learning 在遍历 MDP 上收敛到 (ρ^*, h^*) 。

18 GPU 上的动态规划

大白话

值迭代每个状态更新是**独立的**——可以并行！

GPU 加速

$|S| = 100000$ ，值迭代每次遍历所有状态：

CPU：100000 个状态顺序更新，约 40ms

GPU（1000 核）：并行更新，约 0.4ms

加速 100 倍！（实际受内存带宽限制，约 50 倍）

向量化环境

不仅 planning 可以并行，**数据收集**也可以：

同时运行 1024 个环境，一次收集 1024 个 episode！

这是 Isaac Gym 的核心思想：GPU 原生仿真 + 向量化环境 → 训练速度提升 10000 倍。

表格方法的极限

问题	$ S $	可行？
井字棋	$\sim 10^3$	✓
GridWorld 100×100	10^4	✓
Atari（图像）	$\sim 10^{50000}$	×
围棋	$\sim 10^{170}$	×

当 $|S| > 10^6$ ，即使 GPU 也无法存储值函数表 → 需要**函数逼近**（下周）

19 收敛性理论大总结

算法	收敛性	条件	速度/界
Planning (已知模型)			
值迭代	全局	$\gamma < 1$	$O(\gamma^k)$
策略迭代	全局	有限 MDP	有限步
表格 RL			
Q-learning	全局 w.p.1	无限访问	$O(1/k)$
SARSA	全局 w.p.1	GLIE	$O(1/k)$
TD(λ)	全局	On-policy	依赖 λ
UCB-VI	Regret 界	Episodic	$\tilde{O}(\sqrt{H^3 SAK})$
线性函数逼近			
Linear TD	收敛	On-policy	放大 $\frac{1}{\sqrt{1-\gamma^2}}$
Linear Q	可能发散	Off-policy	—
GTD2	收敛	Off-policy	$O(1/k)$
LSTD	闭式解	批量	$O(\sqrt{d/n})$
Multi-Agent			
Minimax-Q	纳什均衡	两人零和	$O(\gamma^k)$
独立 Q	可能不收敛	一般和	—

神奇结果

本讲核心信息:

1. **MDP = Bandit + 状态转移**, 从单步决策到序贯决策
2. **Bellman 方程**: $V^* = T^*V^*$, 收缩映射保证唯一解
3. **UCB-VI**: 把上周的 UCB 推广到 MDP, Regret $\tilde{O}(\sqrt{H^3 SAK})$
4. **Linear TD**: 投影 Bellman 方程, 误差被 $(1 - \gamma^2)^{-1/2}$ 放大
5. **Deadly Triad**: 函数逼近 + Bootstrapping + Off-policy = 发散风险
6. **Simulation Lemma**: 模型误差被 $(1 - \gamma)^{-2}$ 放大
7. **Performance Difference**: 用 A^π 评估新策略 (TRPO/PPO 的理论基础)

课后练习

1. **Bellman 方程**: 3 状态 MDP (状态 1 奖励 1 转到 2, 状态 2 奖励 0 以 0.5 概率留在 2 或转到 3, 状态 3 奖励 2 转到 1), $\gamma = 0.9$ 。解 V^* 。

2. 收缩映射: 证明如果 T 是 γ -收缩, 则 T^n 是 γ^n -收缩。
3. 收敛速度: $\gamma = 0.99$ 时, 值迭代需要多少次迭代将误差从 100 减到 0.01?
4. Linear TD 误差: $\gamma = 0.95$, 最佳线性逼近误差为 1。TD(0) 的误差上界是多少?
5. Simulation Lemma: $\gamma = 0.99$, 模型误差 $\epsilon_P = \epsilon_R = 0.01$, $R_{max} = 1$ 。策略次优性上界是多少?

答案提示

1. 解方程组: $V(1) = 1 + 0.9V(2)$, $V(2) = 0.9(0.5V(2) + 0.5V(3))$, $V(3) = 2 + 0.9V(1)$ 。
2. $\|T^n V - T^n V'\| \leq \gamma \|T^{n-1} V - T^{n-1} V'\| \leq \dots \leq \gamma^n \|V - V'\|$ 。
3. $100 \cdot 0.99^k < 0.01$, 即 $0.99^k < 10^{-4}$, $k > \frac{\ln 10^4}{\ln(1/0.99)} \approx 916.42$, 因此至少需要 917 次。
4. $\frac{1}{\sqrt{1-0.95^2}} = \frac{1}{\sqrt{0.0975}} = 3.2$ 。误差上界 ≤ 3.2 。
5. $\frac{2}{0.01^2}(0.01 + 0.99 \times 0.01 \times 100) = 20000 \times 1.01 \approx 20200$ 。